

CHAPTER 3 – PART 3

SOLVING PROBLEMS BY SEARCHING

INTRODUCTION TO ARTIFICIAL INTELLIGENT (AI)



REV: 19.03.2024

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Informed (Heuristic) Search

- Unlike uninformed search algorithms, **informed (heuristic) search** algorithms consider problem-specific knowledge; thus, they can find solutions more efficiently.
- Informed search algorithms use the **best-first search** approach. That is; in the frontier, the nodes to be expanded are chosen according to **an evaluation function $f(n)$** . The implementation of best-first search is very similar to that for uniform-cost search, except it uses the **evaluation function f** instead of **path cost g** to order the priority queue.

$$f(n) = ?$$

n : node



The choice of f determines the search strategy.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Heuristic Function

Evaluation functions $f(n)$ include a heuristic function $h(n)$.

$g(n)$



The cost of the path from the initial node to the node n .

$h(n)$



An estimated cost of the cheapest path from the state at node n to a goal state

We will deal with heuristic functions, which are

- non-negative,
- problem-specific.
- $h(n) = 0$ if n is a goal node

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Here, two best-first search strategies are introduced:

- Greedy best-first search
- A* (A-star) search

Greedy best-first
search



$$f(n) = h(n)$$

A* search



$$f(n) = g(n) + h(n)$$

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Greedy Best-First Search

Greedy (best-first) search tries to expand the node that is closest to the goal to reach a solution quickly. Thus, it evaluates nodes using only the heuristic function; that is

$$f(n) = h(n)$$

The values of heuristic function cannot be computed from the problem description itself.

For example:

h_{SLD}



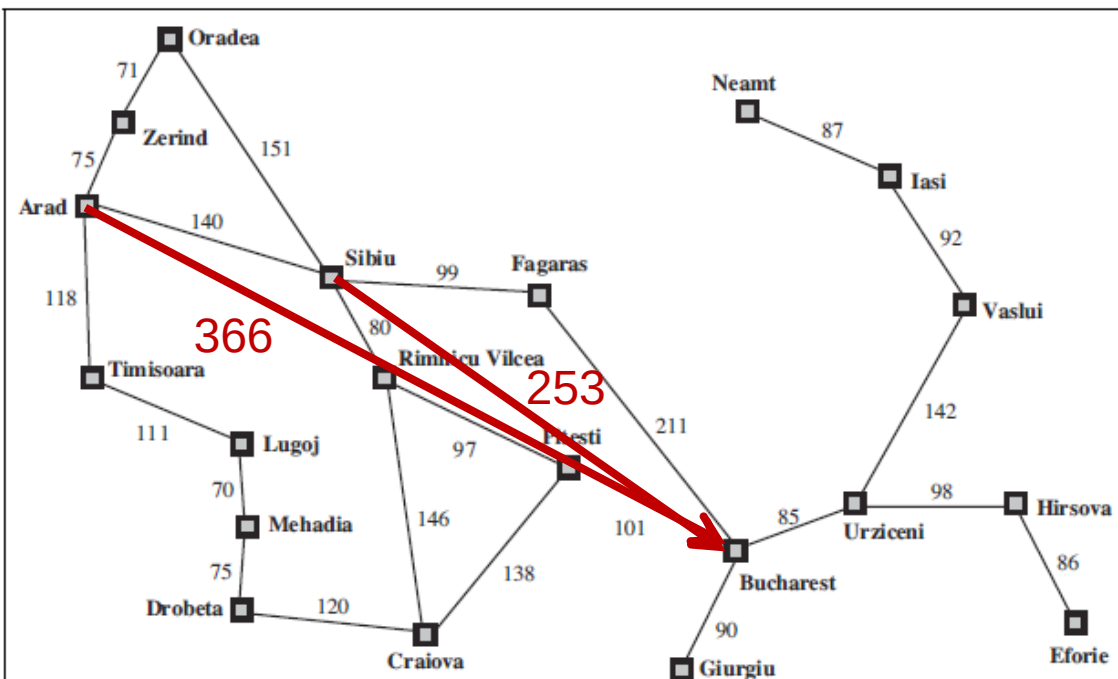
The **Straight Line Distance Heuristic** is one of the most used heuristic functions.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

AI

The Straight-Line Distance Heuristic



A simplified road map of part of Romania.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

The Straight Line
Distance Heuristic

h_{SLD}

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



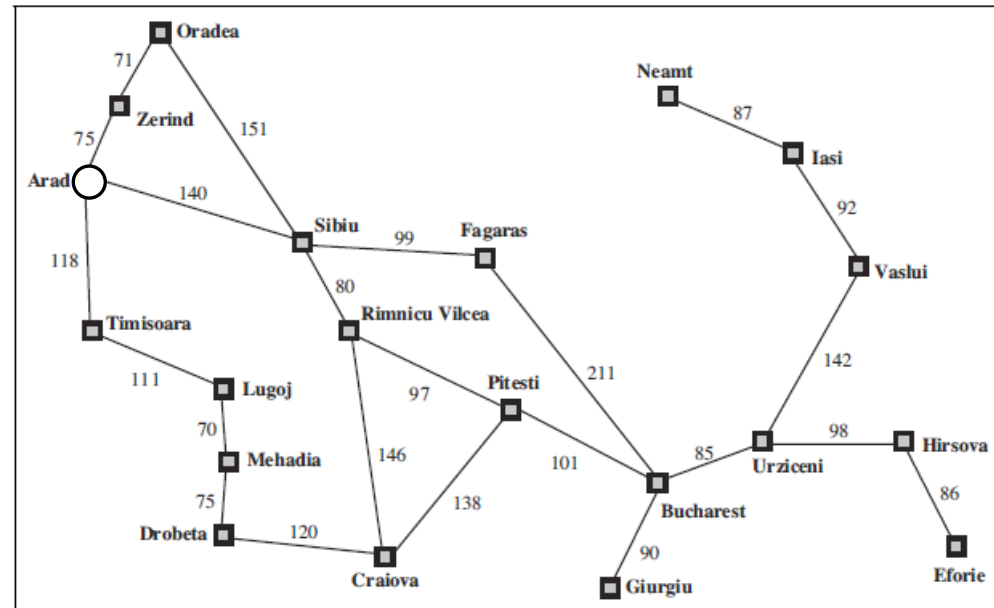
The **tree-search** algorithm for **greedy best-first search** is given below.

```
function GREEDY-BEST-FIRST-SEARCH-TREE (problem) returns a solution, or failure
- node ← a node with STATE = problem.INITIAL-STATE() /*including heuristic function */
    PATH-COST = 0
- if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
- frontier ← a priority queue ordered by  $h(n)$ , with the root node
- loop do
    - if EMPTY( frontier) then return failure
    - node ← POP( frontier ) /* chooses node with min heuristic function in frontier */
    - for each action in problem.ACTIONS(node.STATE) do
        - child ← CHILD-NODE(problem, node, action)
        - if problem.GOAL-TEST(child .STATE) then return SOLUTION(child )
        - frontier ← INSERT(child , frontier )
```

INTRODUCTION TO ARTIFICIAL INTELLIGENT



[Arad₃₆₆]

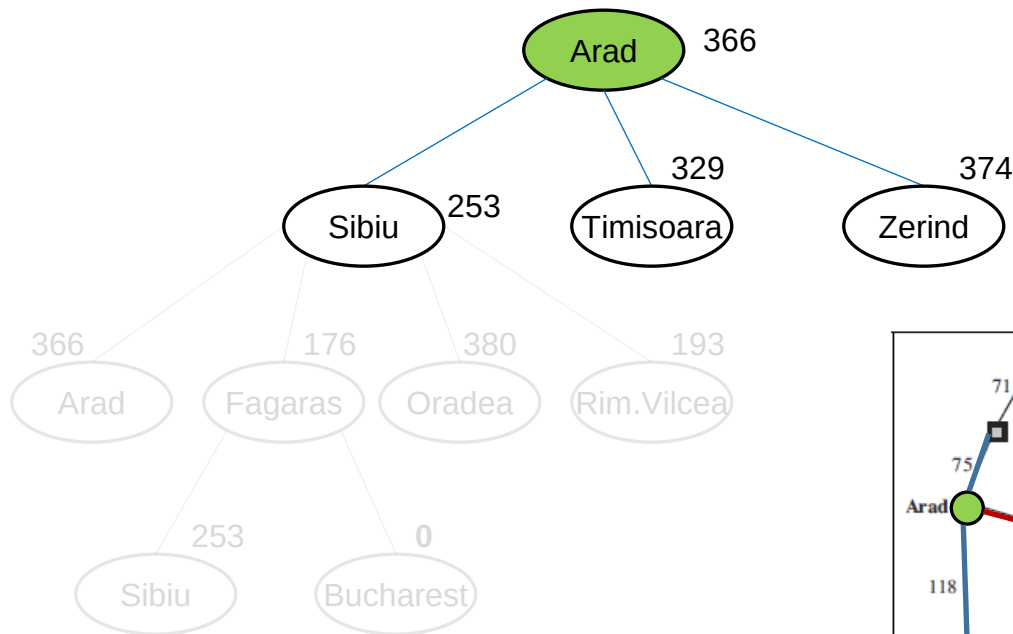


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

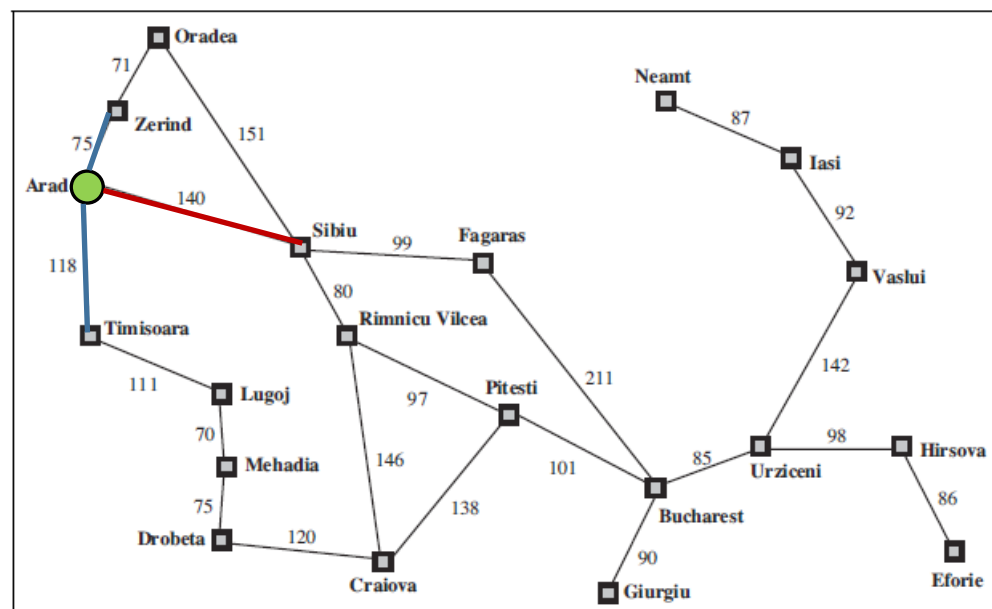


Ex : The Romania Map



[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]

QUEUE

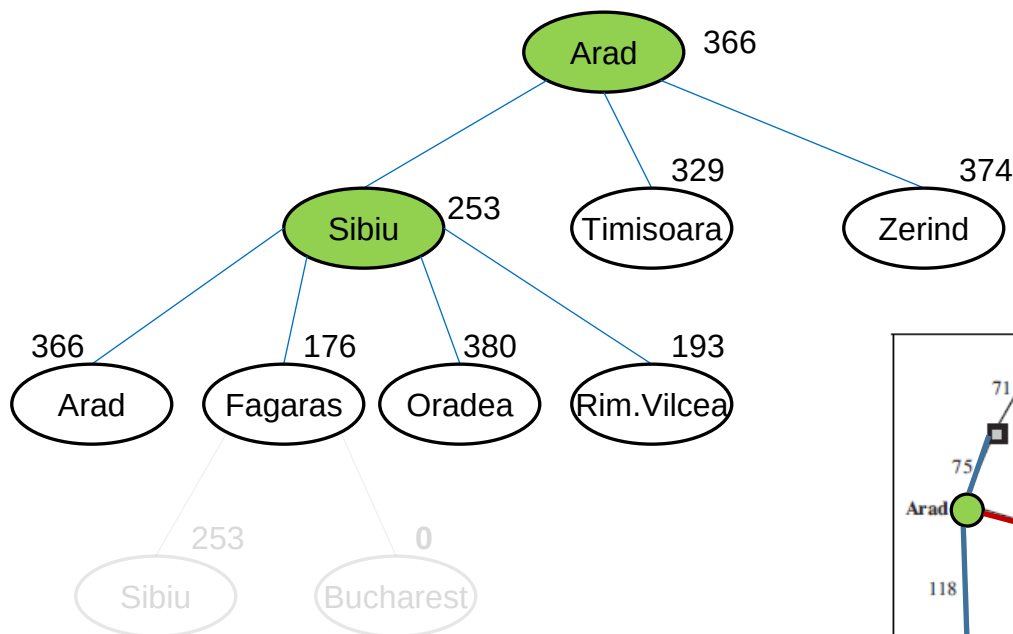


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

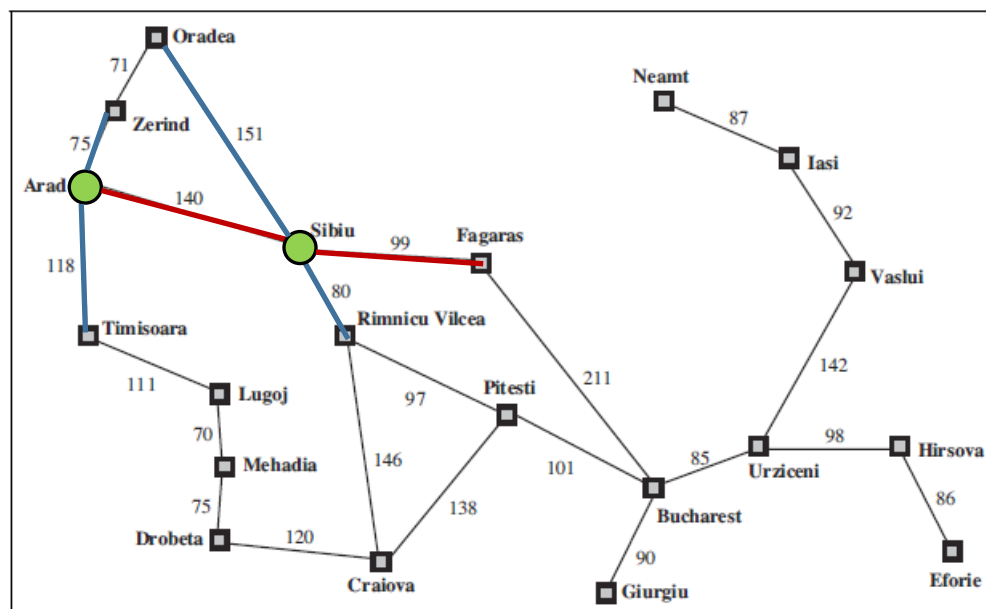


Ex : The Romania Map



QUEUE

[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Fagaras₁₇₆, Rim.Vilcea₁₉₃, Timisoara₃₂₉, Arad₃₆₆,
Zerind₃₇₄, Oradea₃₈₀]

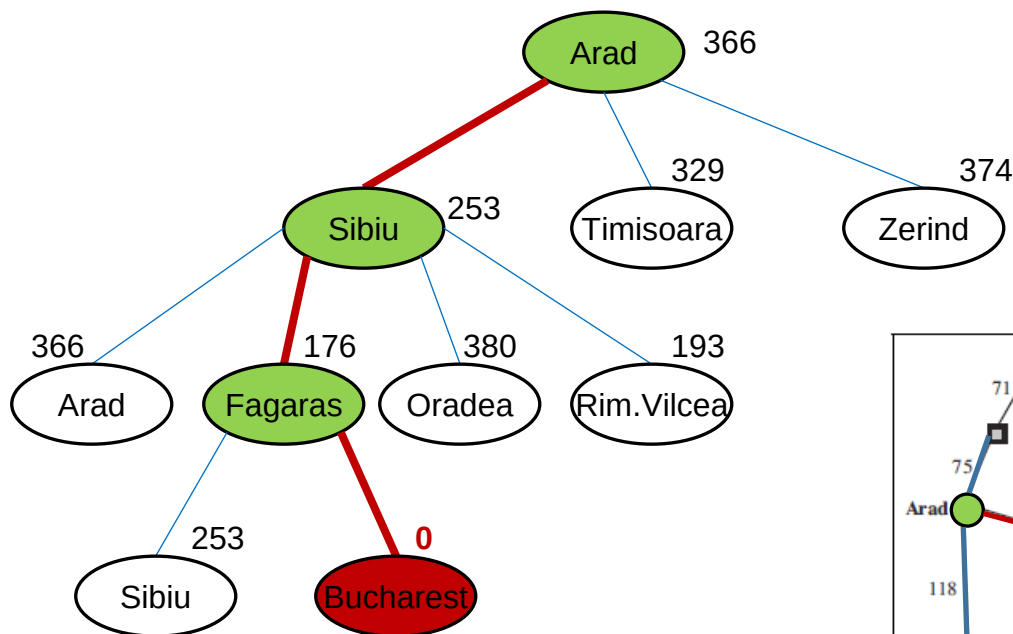


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

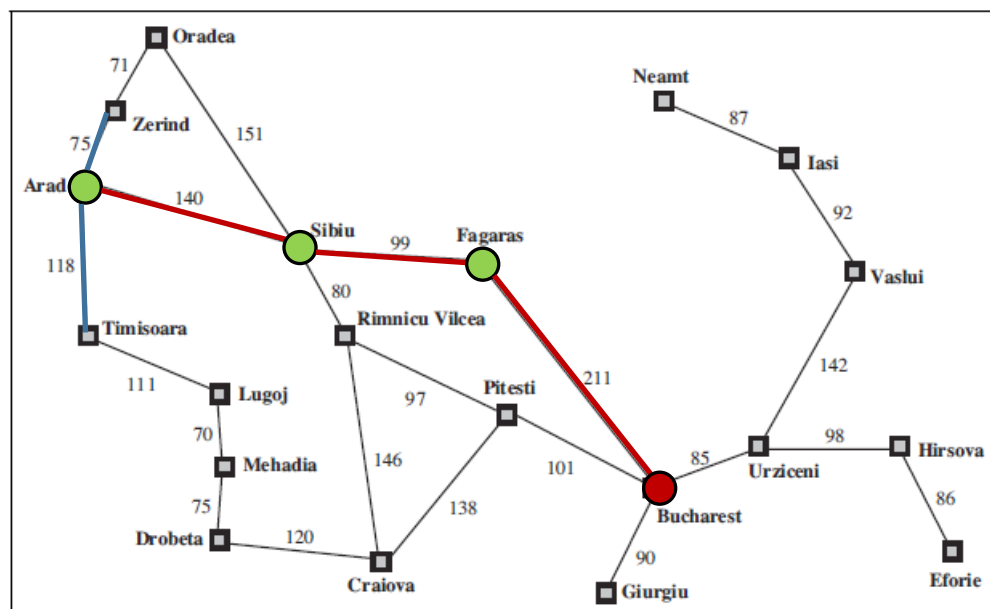


Ex : The Romania Map



QUEUE

[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Fagaras₁₇₆, Rim.Vilcea₁₉₃, Timisoara₃₂₉, Arad₃₆₆,
Zerind₃₇₄, Oradea₃₈₀]
[Rim.Vilcea₁₉₃, Sibiu₂₅₃, Timisoara₃₂₉, Arad₃₆₆,
Zerind₃₇₄, Oradea₃₈₀]



Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The **graph-search** algorithm for **greedy best-first search** is given below.

```
function GREEDY-BEST-FIRST-SEARCH-GRAPH (problem) returns a solution, or failure
- node ← a node with STATE = problem.INITIAL-STATE /*including heuristic
function*/
```

PATH-COST = 0

- if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
- frontier ← a priority queue ordered by $h(n)$, with the root node
- explored ← an empty set
- loop do
 - if EMPTY(frontier) then return failure
 - node ← POP(frontier) /* chooses node with min heuristic function in frontier */
 - add node.STATE to explored
 - for each action in problem.ACTIONS(node.STATE) do
 - child ← CHILD-NODE(problem, node, action)
 - if child.STATE is not in explored or frontier then
 - if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
 - frontier ← INSERT(child, frontier)

Solving Problems by Searching - 3

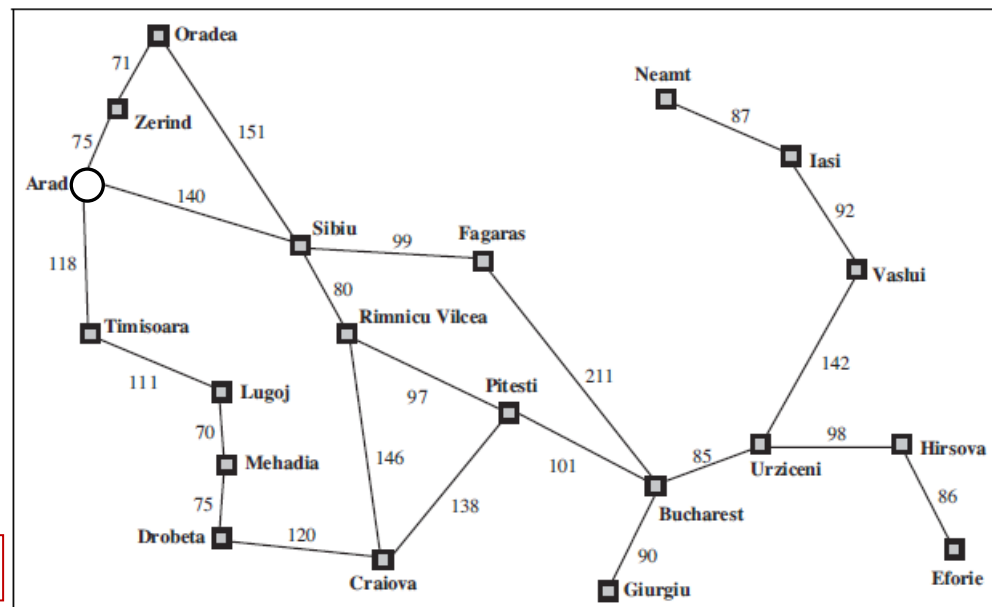
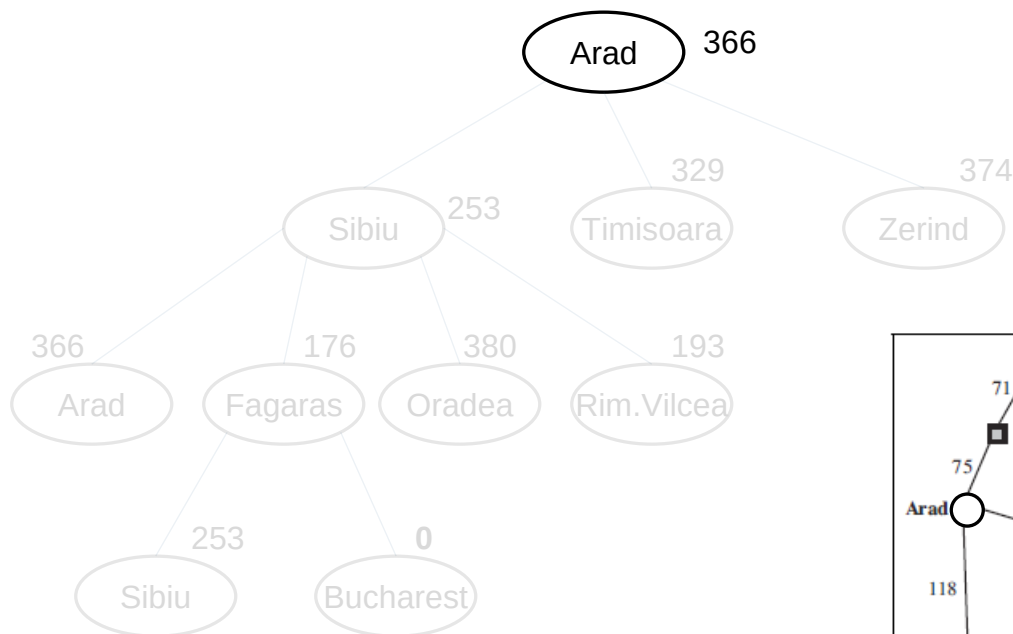
INTRODUCTION TO ARTIFICIAL INTELLIGENT



Ex : The Romania Map

[Arad₃₆₆]

QUEUE



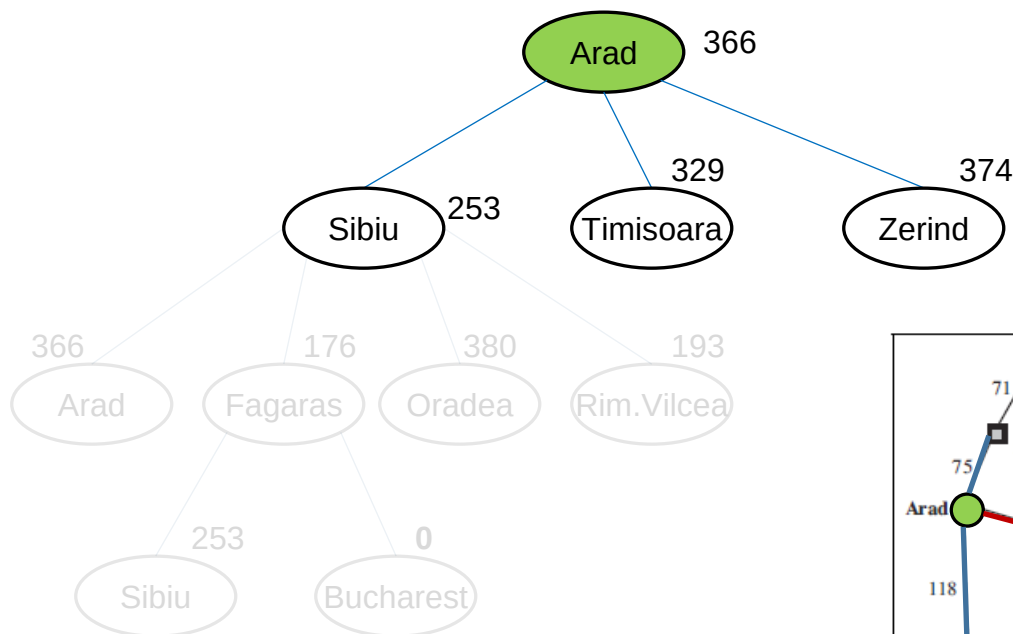
EXPLORED SET:

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



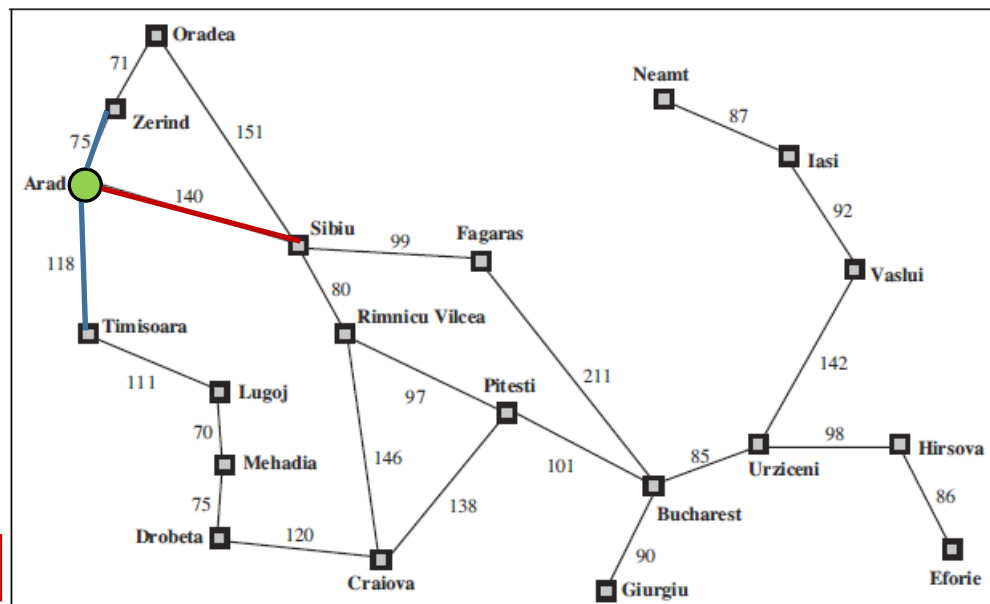
Ex : The Romania Map



[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]

QUEUE

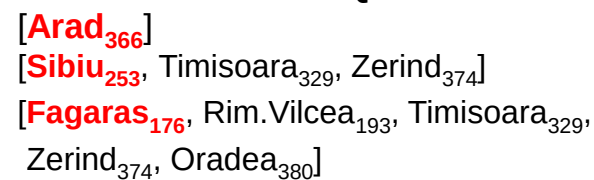
EXPLORED SET: Arad



INTRODUCTION TO ARTIFICIAL INTELLIGENT



QUEUE

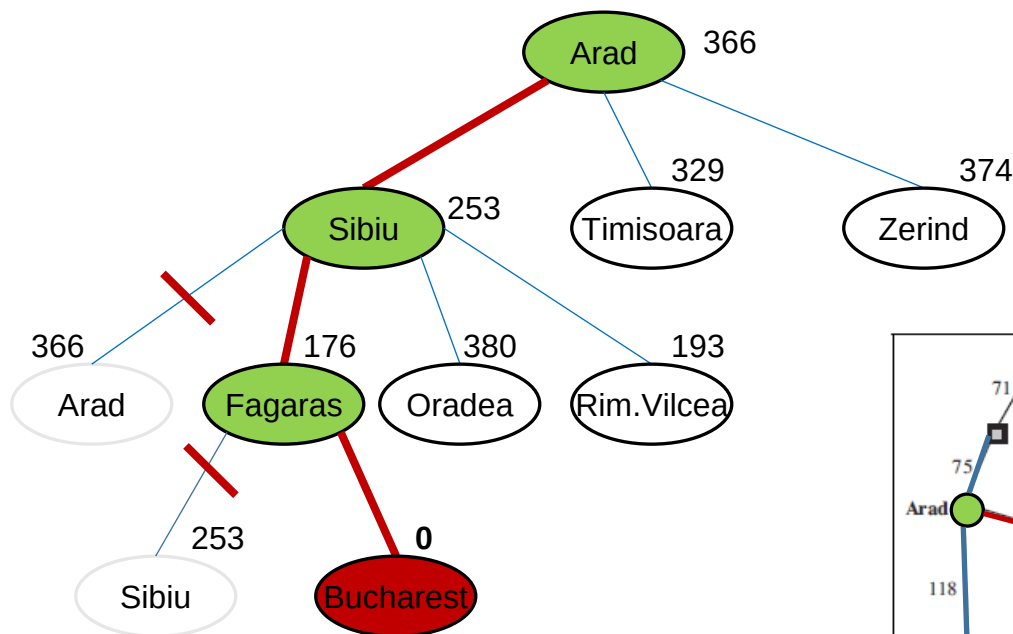


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



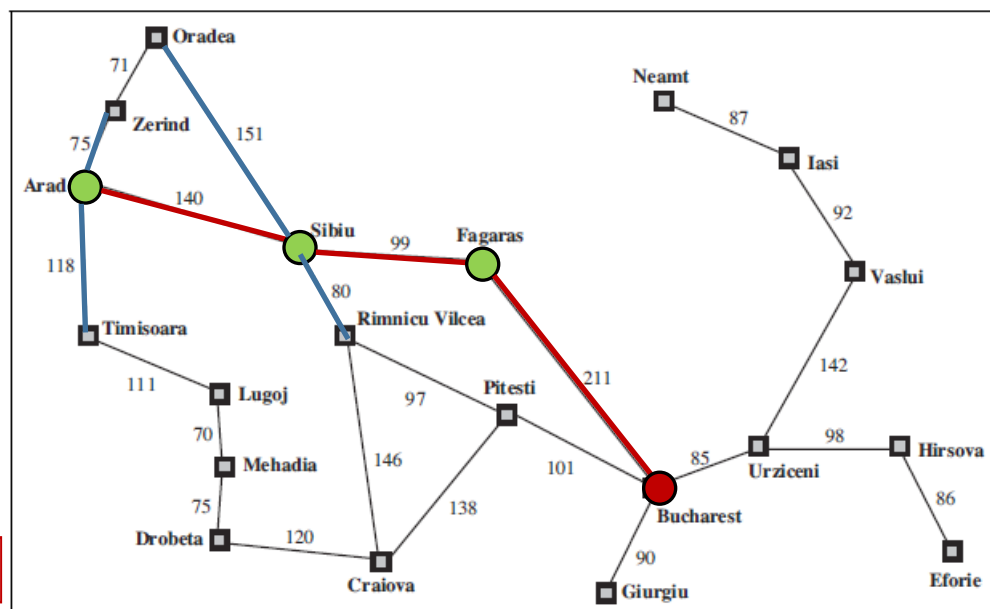
Ex : The Romania Map



QUEUE

[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Fagaras₁₇₆, Rim.Vilcea₁₉₃, Timisoara₃₂₉,
Zerind₃₇₄, Oradea₃₈₀]
[Rim.Vilcea₁₉₃, Timisoara₃₂₉,
Zerind₃₇₄, Oradea₃₈₀]

EXPLORED SET: Arad, Sibiu, Fagaras

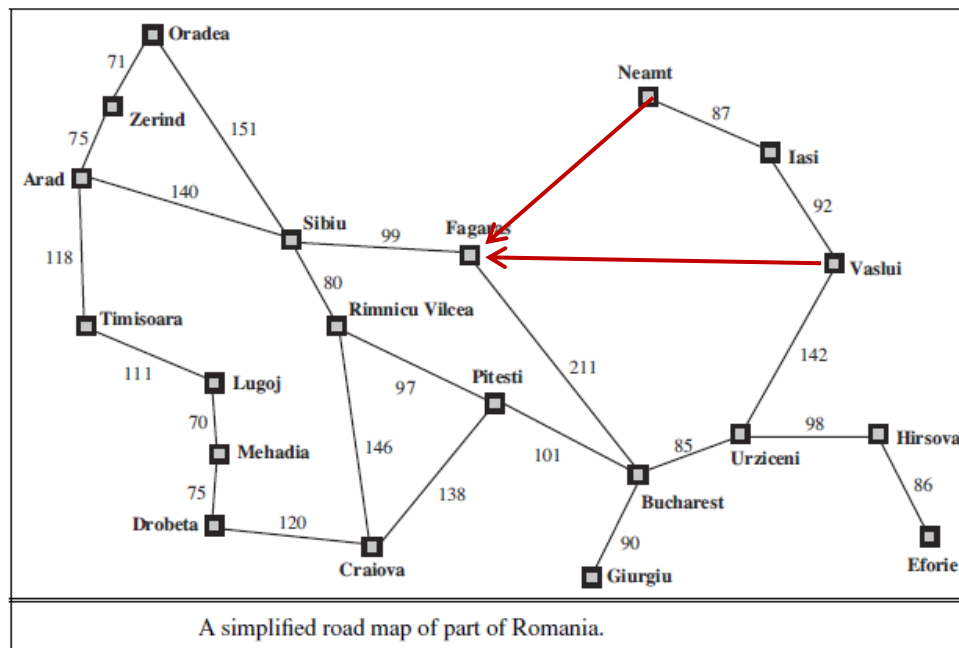


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Greedy Best-First search algorithm's performance



A simplified road map of part of Romania.

- Greedy best-first tree search is **incomplete** (Consider the solution from Iasi to Fagaras) for tree-search, **complete** (when the state space is finite) for graph-search.
- Greedy best-first search is **not optimal**.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Greedy Best-First search algorithm's performance

- The **worst-case time complexity and space complexity** is $O(b^m)$, where m is the maximum depth of the search space.
- However, with a good heuristic function, the number of searched nodes can be reduced substantially.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



A* Search

A* (A star) search evaluates nodes by combining the path with the heuristic function:

$$f(n) = g(n) + h(n)$$

For example:

h_{SLD}



The Straight Line Distance Heuristic is one of the most used heuristic functions.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The **tree-search** algorithm for **A* search** is given below.

```
function A-STAR-SEARCH -TREE (problem) returns a solution, or failure
- node ← a node with STATE = problem.INITIAL-STATE() /*including heuristic
function*/
    PATH-COST = 0
- frontier ← a priority queue ordered by  $f(n)$ , with the root node
- loop do
-   if EMPTY( frontier) then return failure
-   node ← POP( frontier ) /*chooses node with min evaluation function in frontier */
-   if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
-   for each action in problem.ACTIONS(node.STATE) do
-     child ← CHILD-NODE(problem, node, action)
-     frontier ← INSERT(child , frontier )
```

Solving Problems by Searching - 3

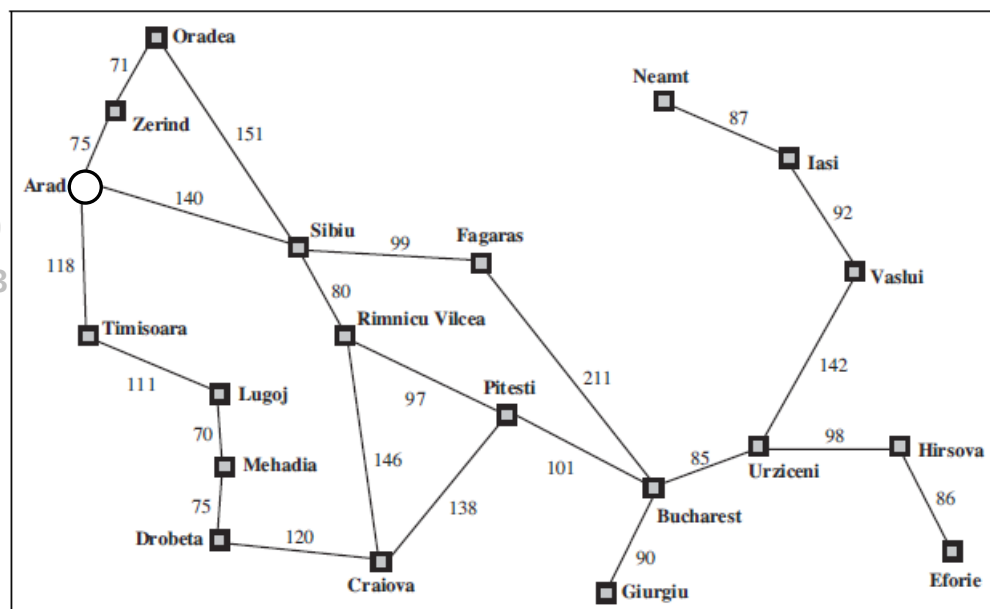
INTRODUCTION TO ARTIFICIAL INTELLIGENT



Ex : The Romania Map

QUEUE

[Arad₃₆₆]

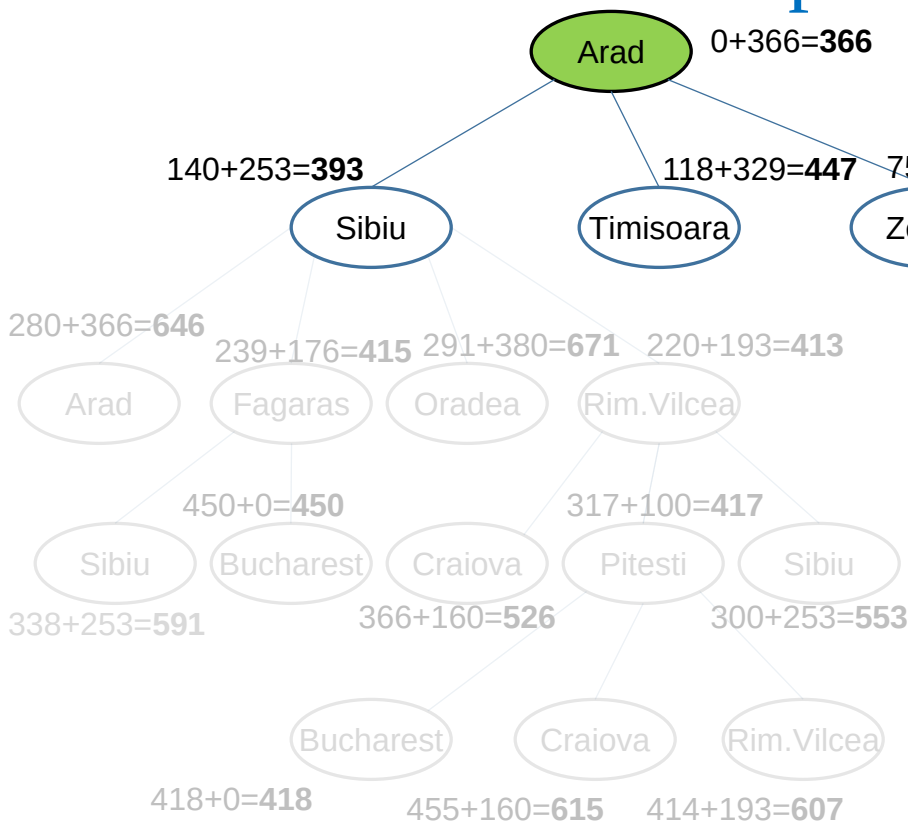


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

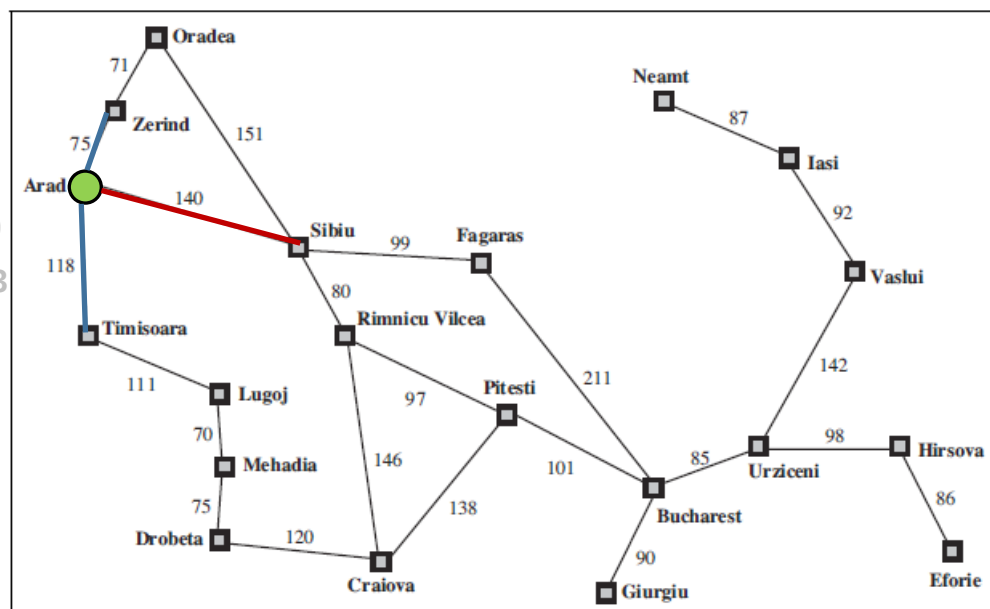


Ex : The Romania Map



[Arad₃₆₆]
[Sibiu₃₉₃, Timisoara₄₄₇, Zerind₄₄₉]

QUEUE

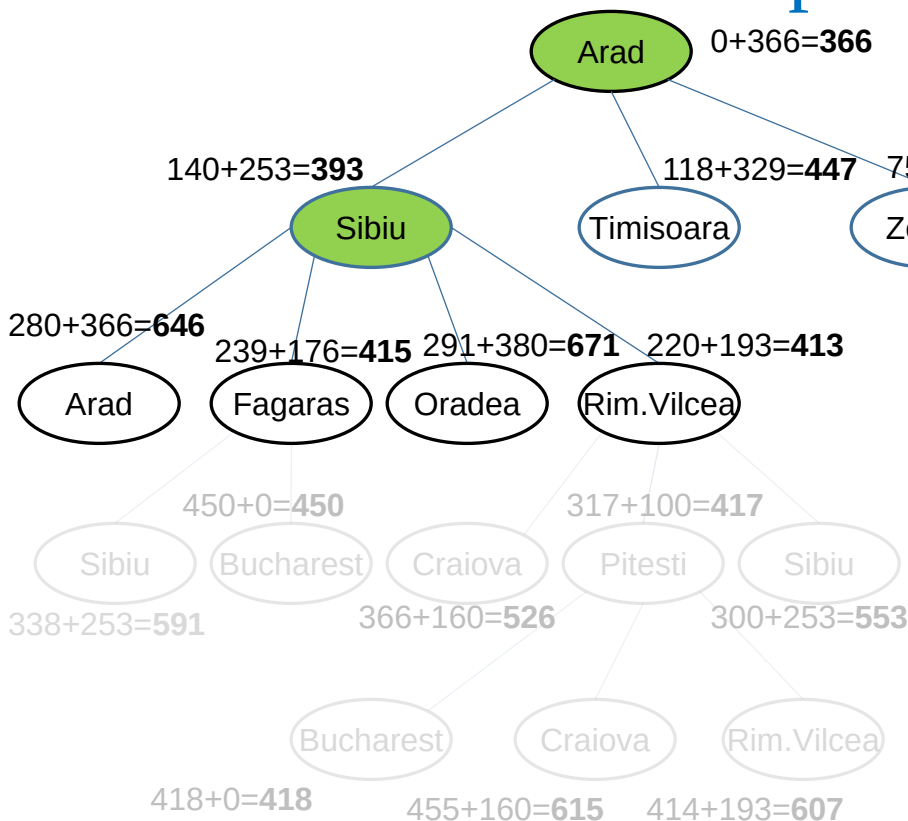


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

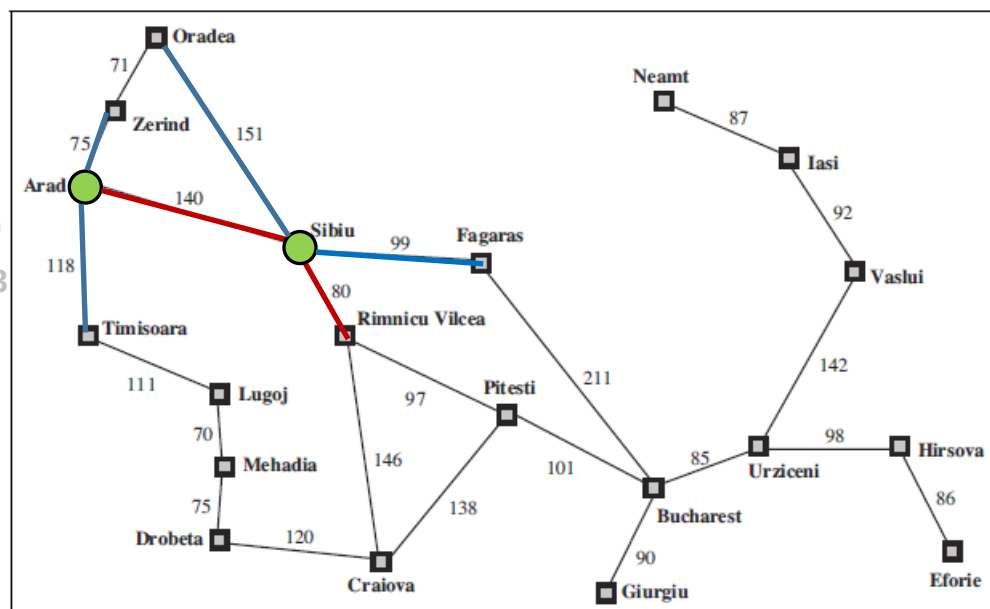


Ex : The Romania Map



QUEUE

[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉,
Arad₆₄₆, Oradea₆₇₁]

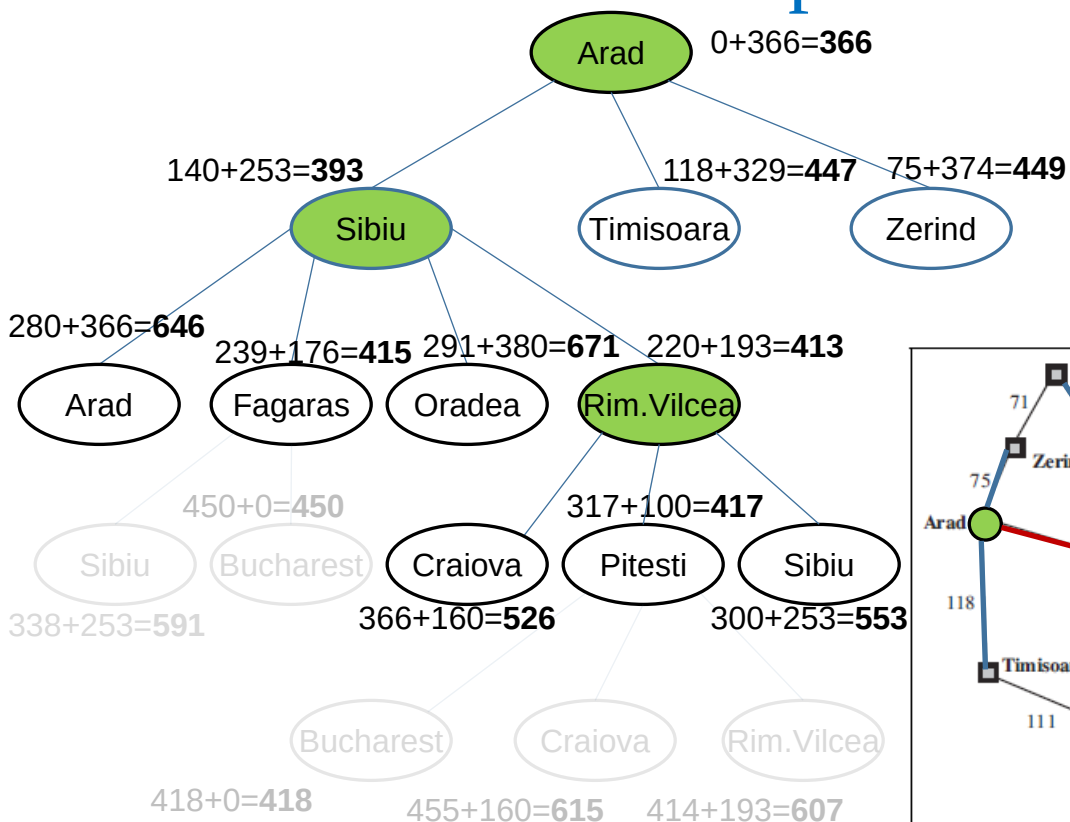


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

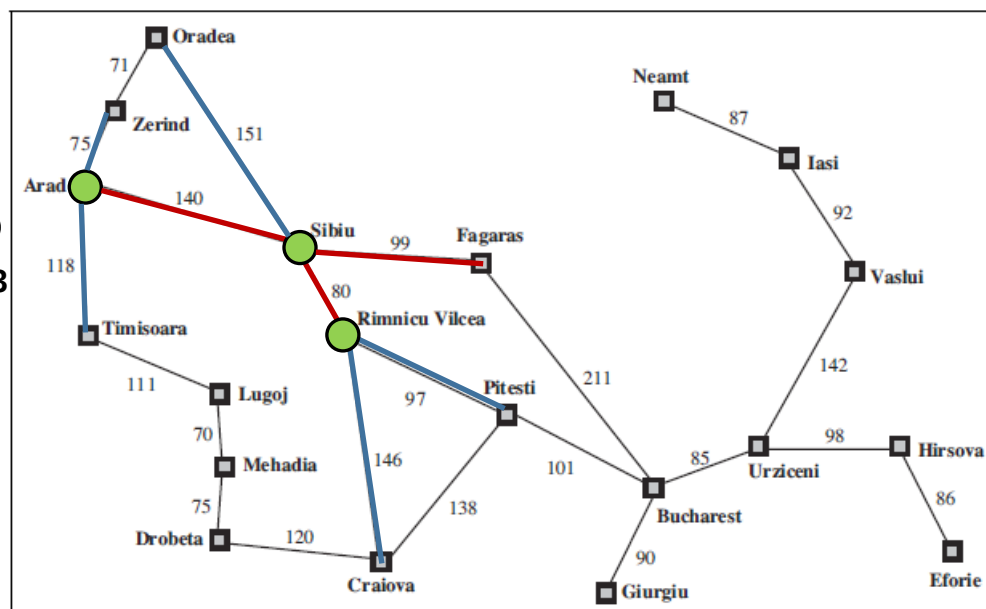


Ex : The Romania Map



QUEUE

[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉, Arad₆₄₆, Oradea₆₇₁]
[Fagaras₄₁₅, Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Craiova₅₂₆, Sibiu₅₅₃, Arad₆₄₆, Oradea₆₇₁]

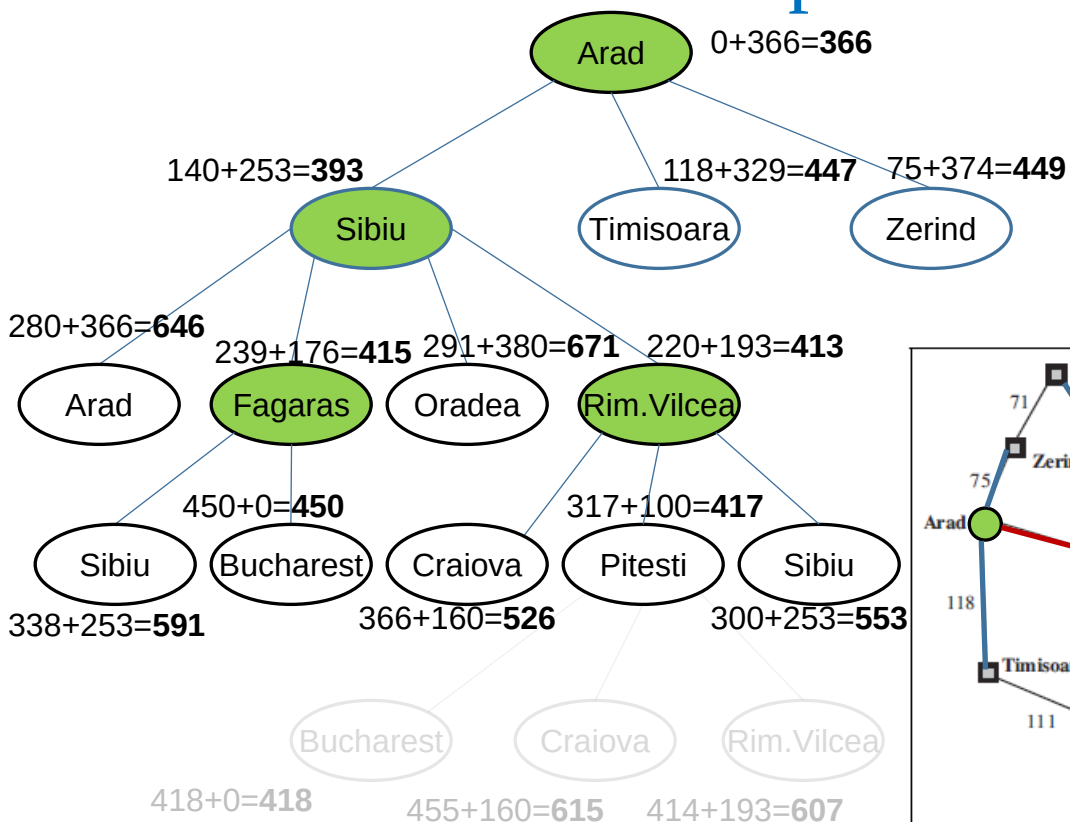


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

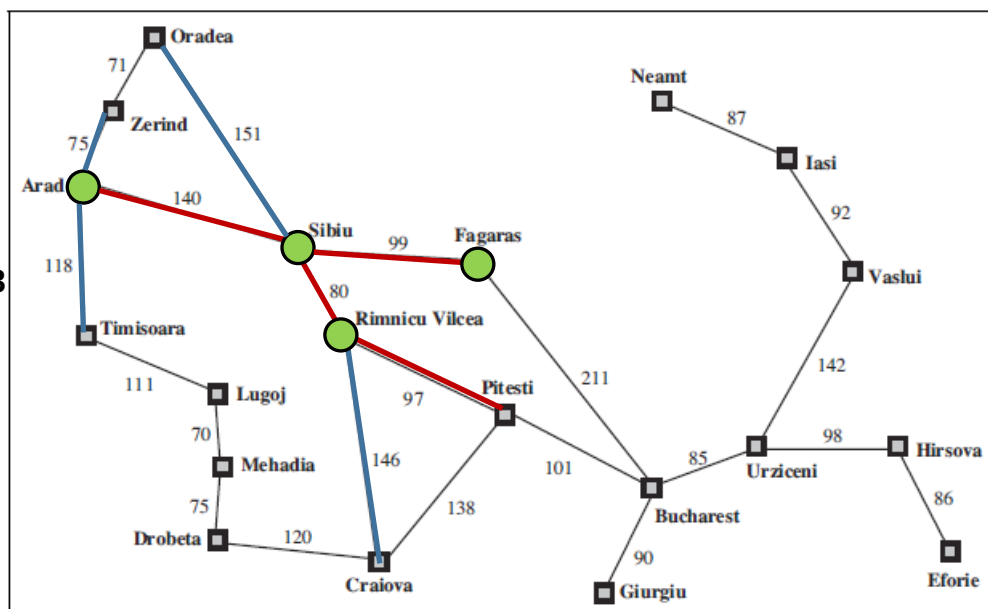


Ex : The Romania Map



QUEUE

[Arad₃₆₆]
 [Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
 [Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉, Arad₆₄₆, Oradea₆₇₁]
 [Fagaras₄₁₅, Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Craiova₅₂₆, Sibiu₅₅₃, Arad₆₄₆, Oradea₆₇₁]
 [Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Bucharest₄₅₀, Craiova₅₂₆, Sibiu₅₅₃, Sibiu₅₉₁, Arad₆₄₆, Oradea₆₇₁]

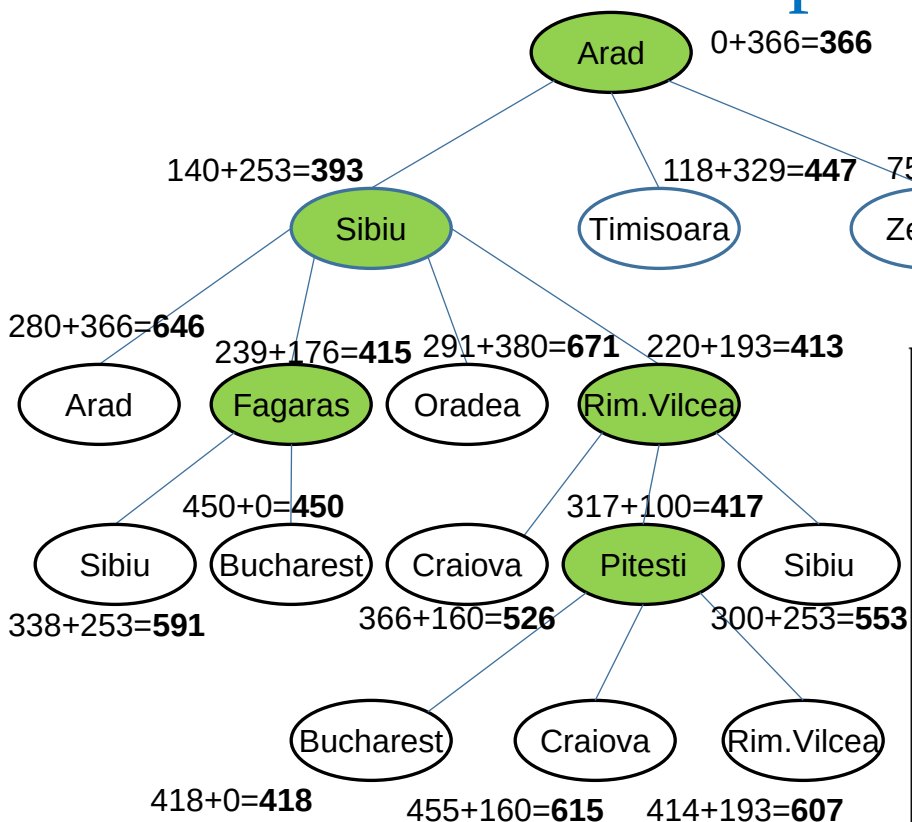


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

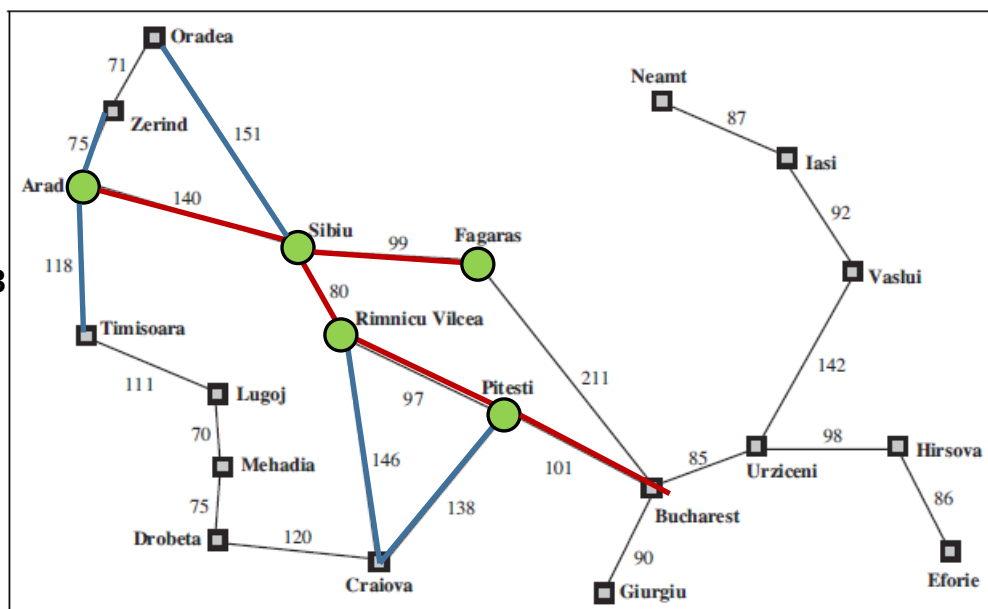


Ex : The Romania Map



QUEUE

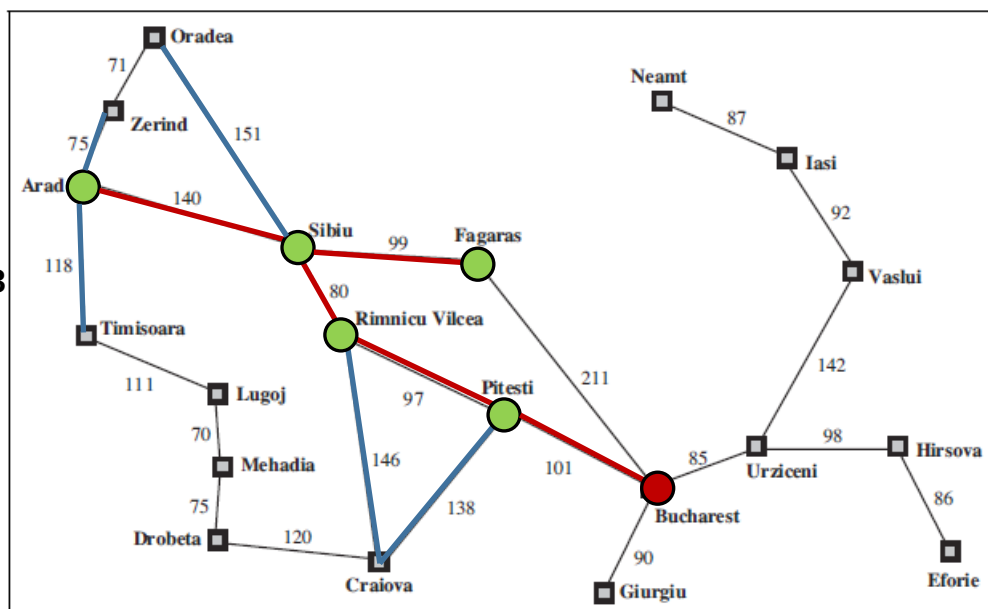
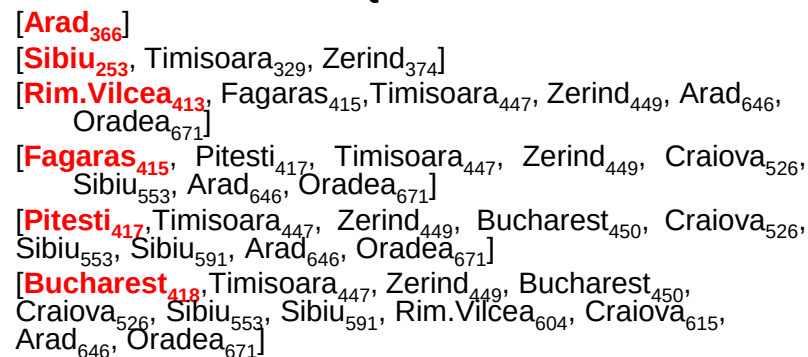
[Arad₃₆₆]
 [Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
 [Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉, Arad₆₄₆, Oradea₆₇₁]
 [Fagaras₄₁₅, Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Craiova₅₂₆, Sibiu₅₅₃, Arad₆₄₆, Oradea₆₇₁]
 [Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Bucharest₄₅₀, Craiova₅₂₆, Sibiu₅₅₃, Sibiu₅₉₁, Arad₆₄₆, Oradea₆₇₁]
 [Bucharest₄₁₈, Timisoara₄₄₇, Zerind₄₄₉, Bucharest₄₅₀, Craiova₅₂₆, Sibiu₅₅₃, Sibiu₅₉₁, Rim.Vilcea₆₀₄, Craiova₆₁₅, Arad₆₄₆, Oradea₆₇₁]



INTRODUCTION TO ARTIFICIAL INTELLIGENT



QUEUE



Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The **graph-search** algorithm for **A* search** is given below.

```
function A-STAR-SEARCH -GRAPH (problem) returns a solution, or failure
- node ← a node with STATE = problem.INITIAL-STATE()/*including heuristic function*/
  PATH-COST = 0
- frontier ← a priority queue ordered by  $f(n)$ , with the root node
- explored ← an empty set
- loop do
  - if EMPTY( frontier) then return failure
  - node ← POP( frontier ) /*chooses node with min evaluation function in frontier */
  - if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  - add node.STATE to explored
  - for each action in problem.ACTIONS(node.STATE) do
    - child ← CHILD-NODE(problem, node, action)
    - if child .STATE is not in explored or frontier then
      - frontier ← INSERT(child , frontier )
    else if child .STATE is in frontier with higher  $f(n)$  then
      - replace that frontier node with the child
```

Solving Problems by Searching - 3

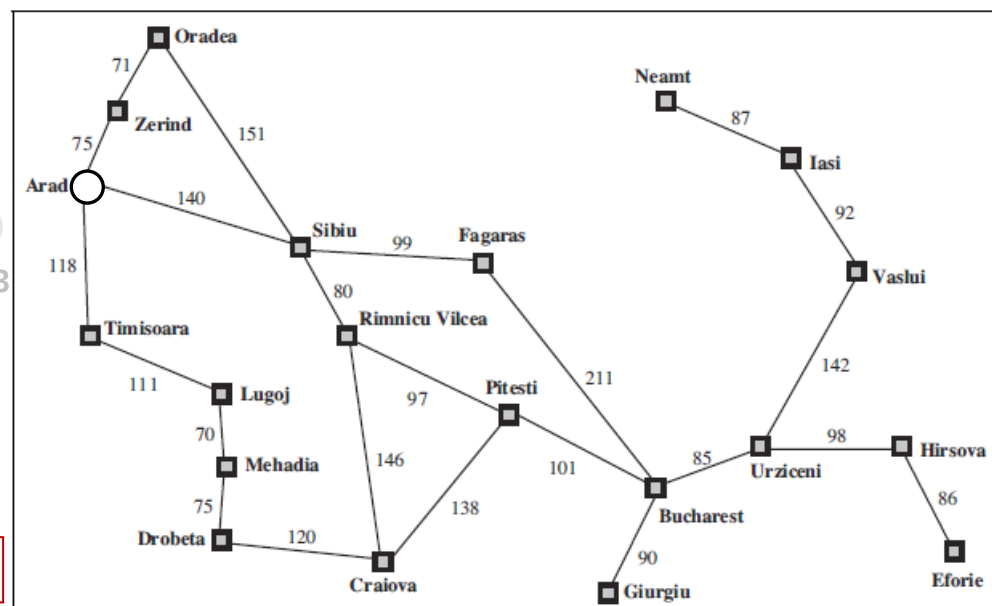
INTRODUCTION TO ARTIFICIAL INTELLIGENT



Ex : The Romania Map

QUEUE

[Arad₃₆₆]



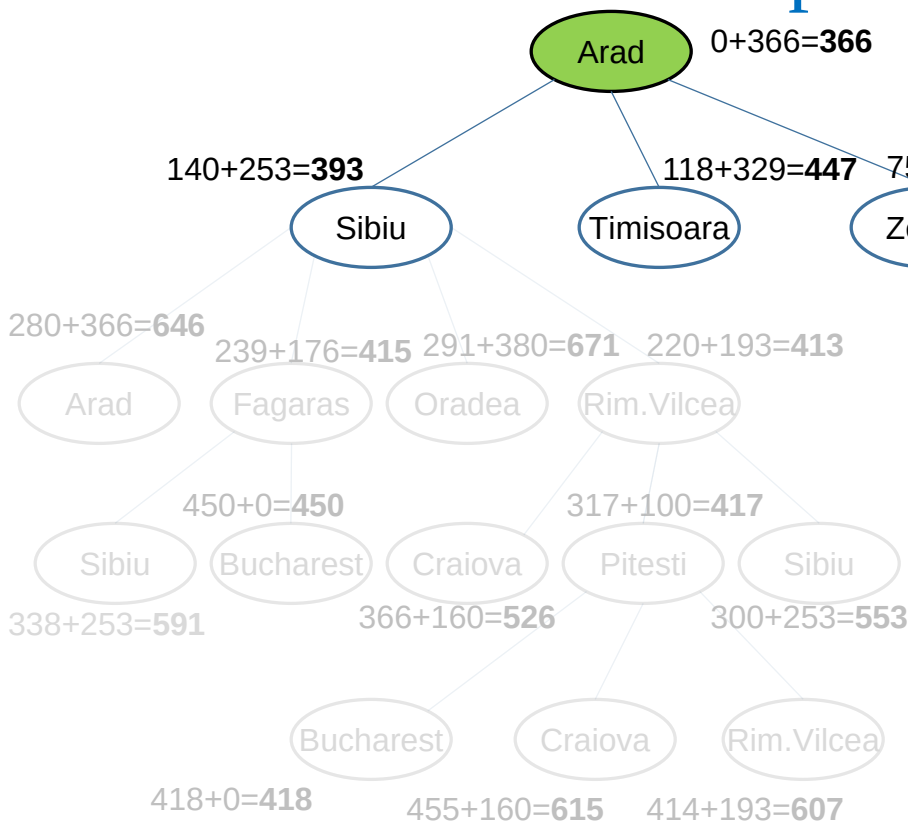
EXPLORED SET:

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



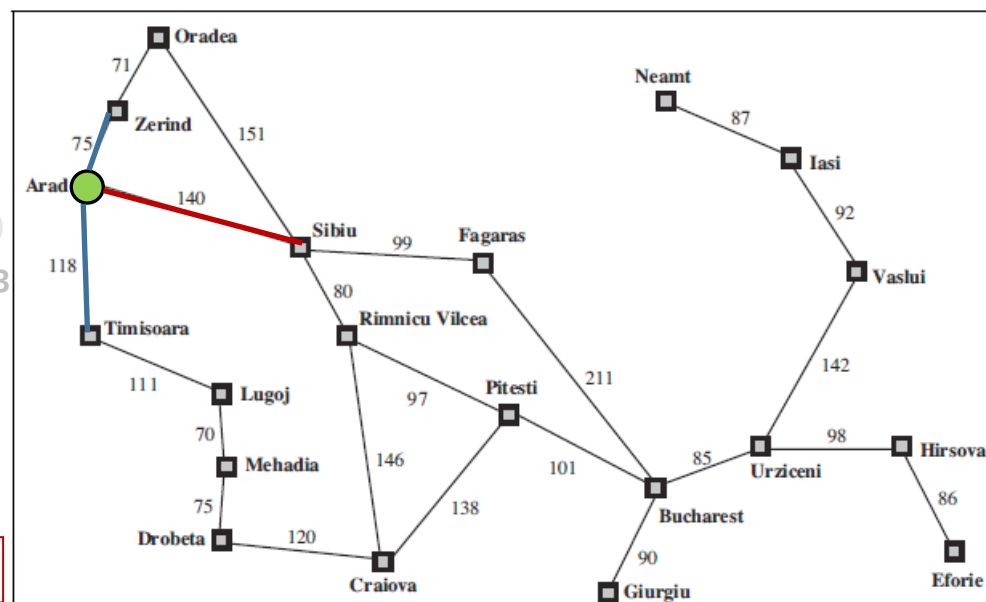
Ex : The Romania Map



EXPLORED SET: Arad

QUEUE

[Arad₃₆₆]
[Sibiu₃₉₃, Timisoara₄₄₇, Zerind₄₄₉]

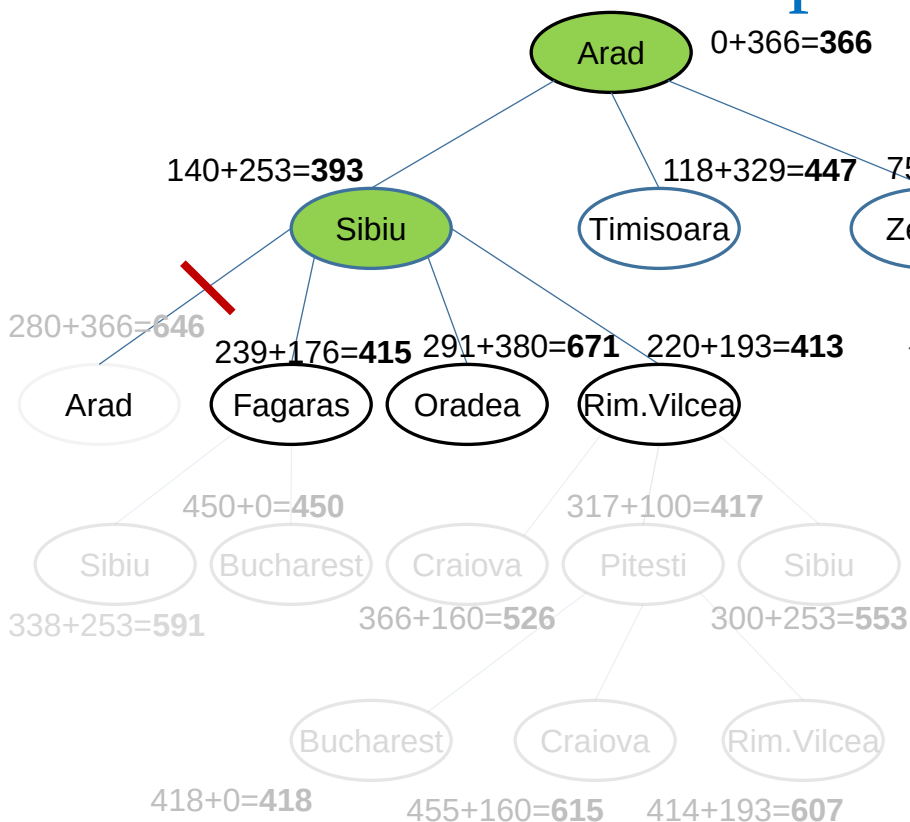


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



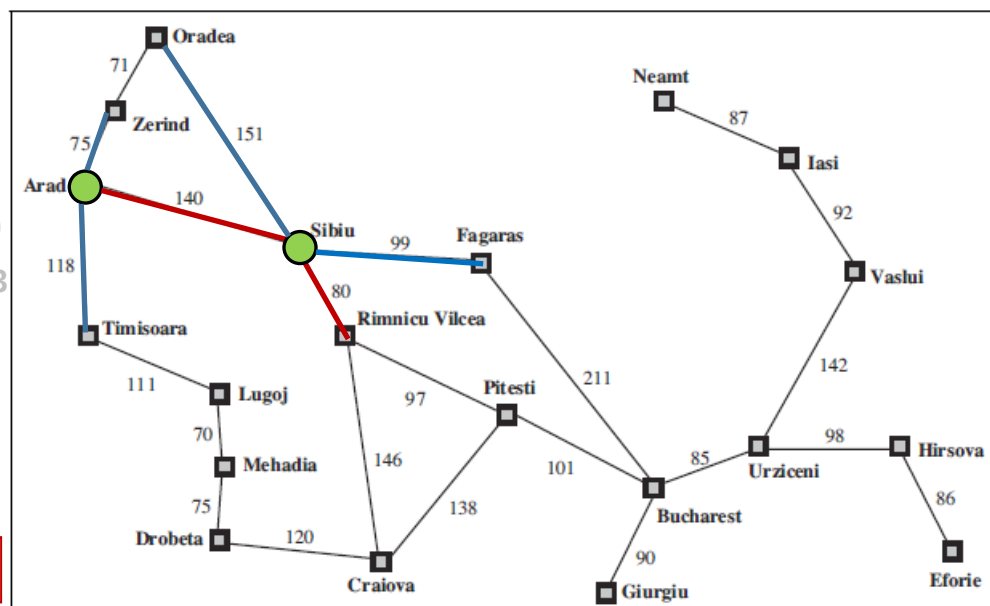
Ex : The Romania Map



EXPLORED SET: Arad, Sibiu

QUEUE

[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉, Oradea₆₇₁]

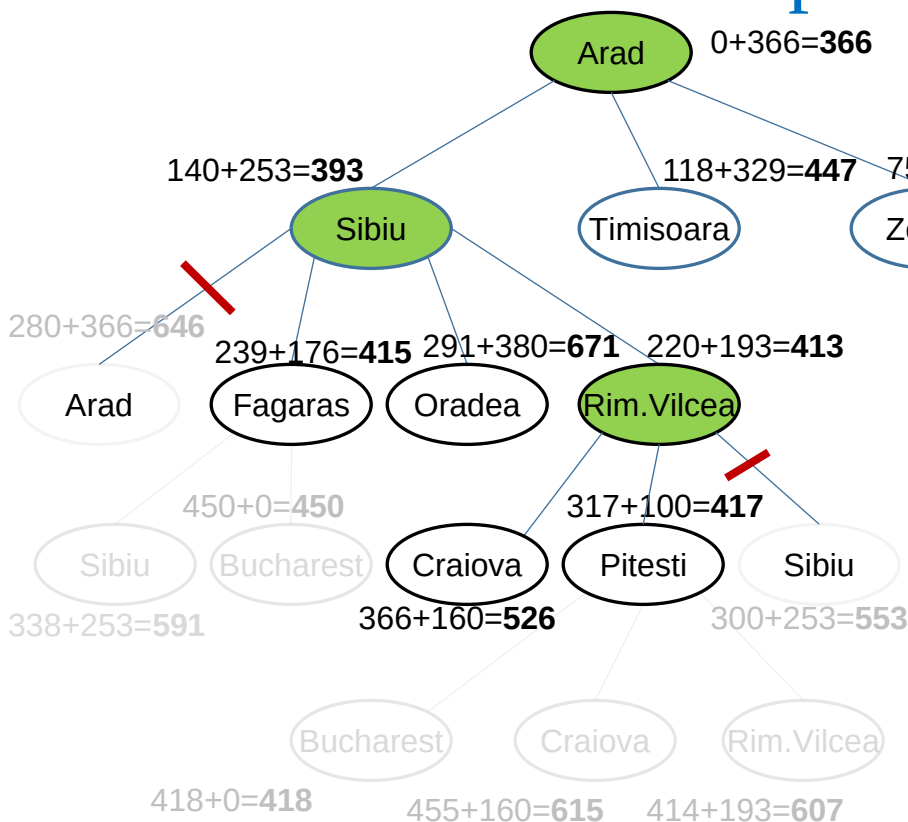


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



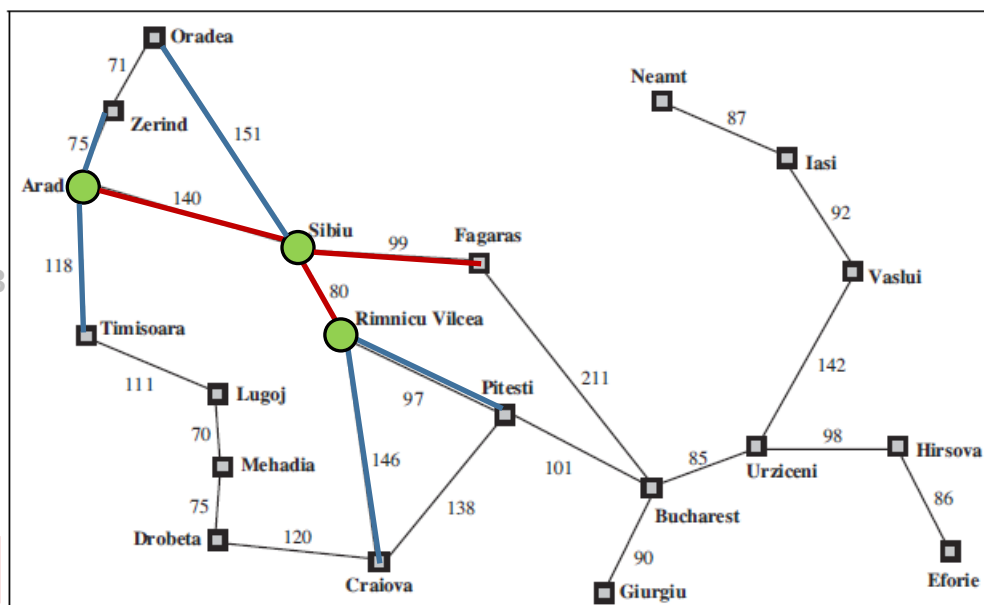
Ex : The Romania Map



EXPLORED SET: Arad, Sibiu, Rim.Vil.

QUEUE

[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉, Oradea₆₇₁]
[Fagaras₄₁₅, Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Craiova₅₂₆, Oradea₆₇₁]

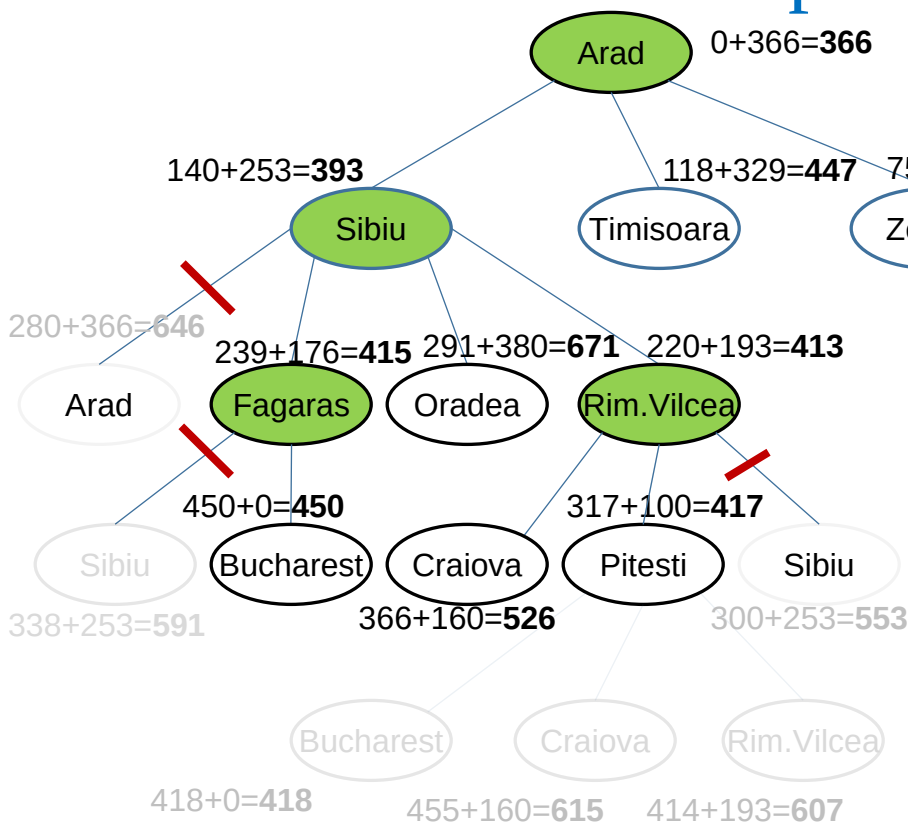


Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



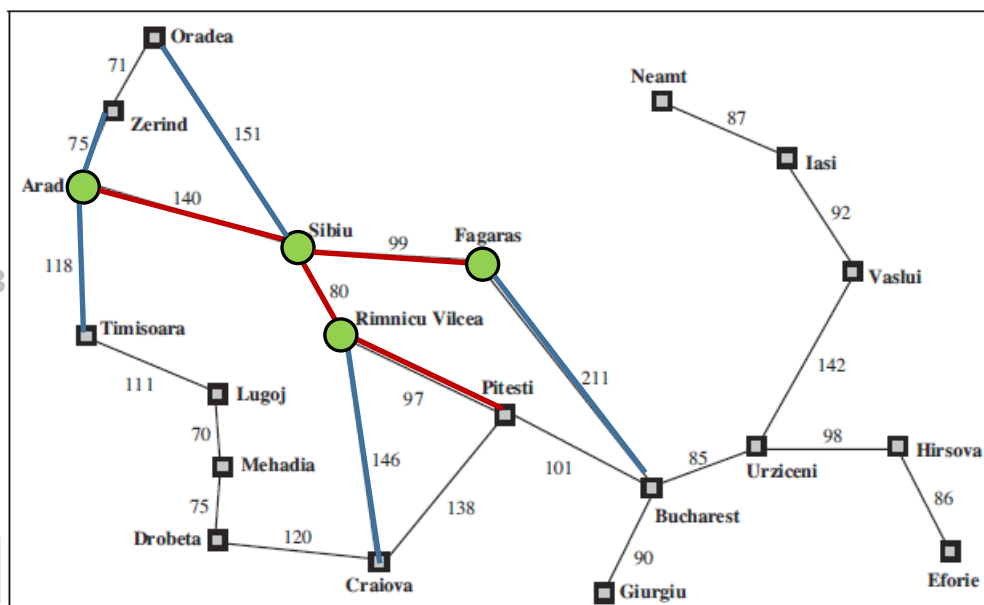
Ex : The Romania Map



EXPLORED SET: Arad, Sibiu, Rim.Vil., Fagaras

QUEUE

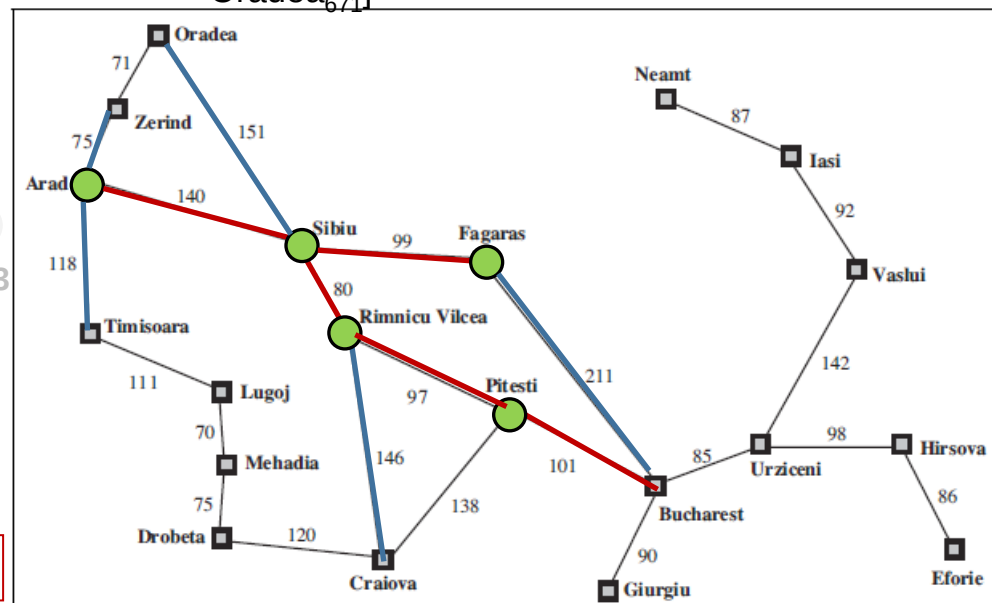
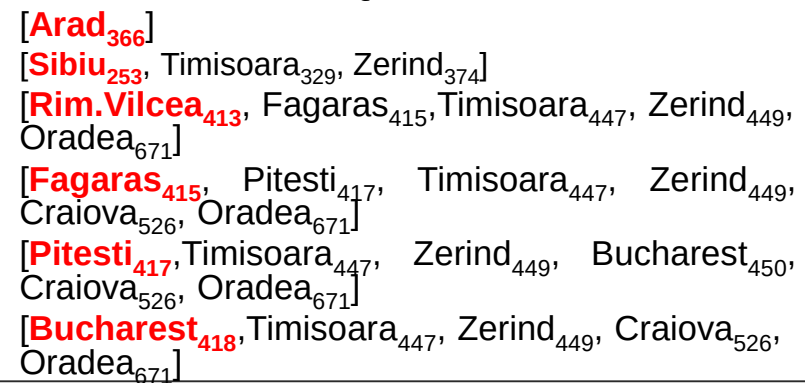
[Arad₃₆₆]
[Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
[Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉, Oradea₆₇₁]
[Fagaras₄₁₅, Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Craiova₅₂₆, Oradea₆₇₁]
[Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Bucharest₄₅₀, Craiova₅₂₆, Oradea₆₇₁]



INTRODUCTION TO ARTIFICIAL INTELLIGENT



QUEUE



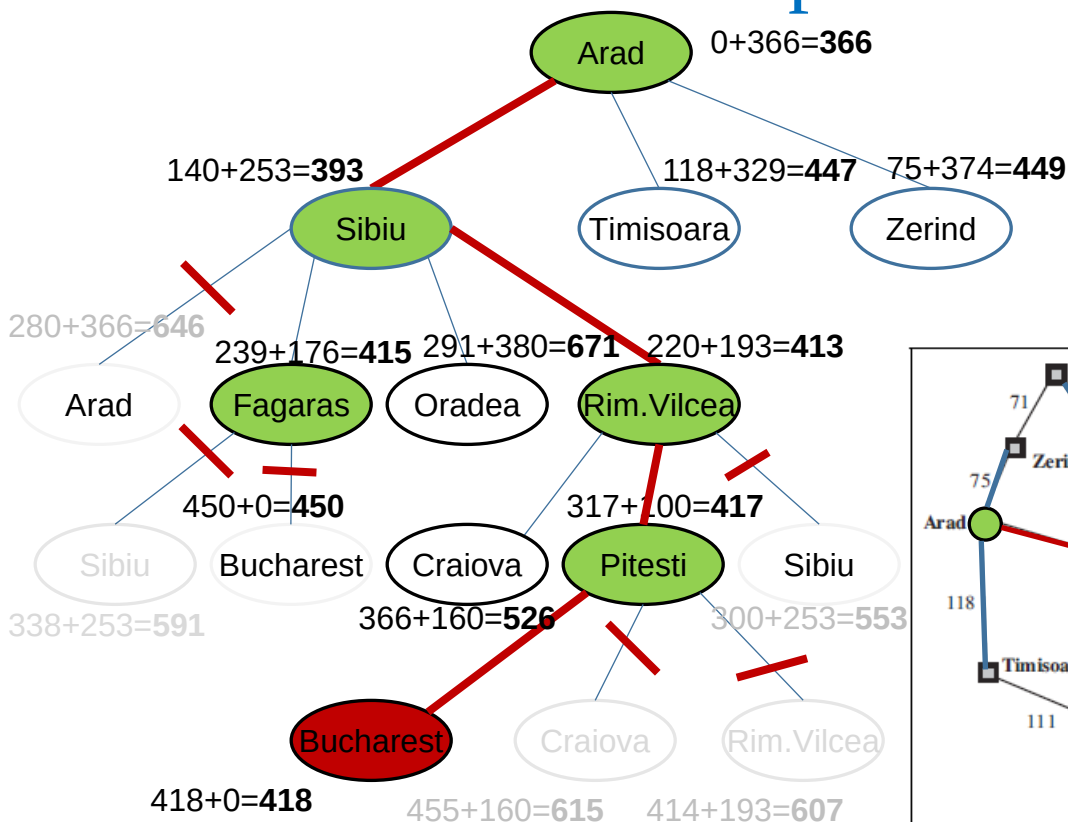
EXPLORED SET: Arad, Sibiu, Rim.Vil., Fagaras, Pitesti

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

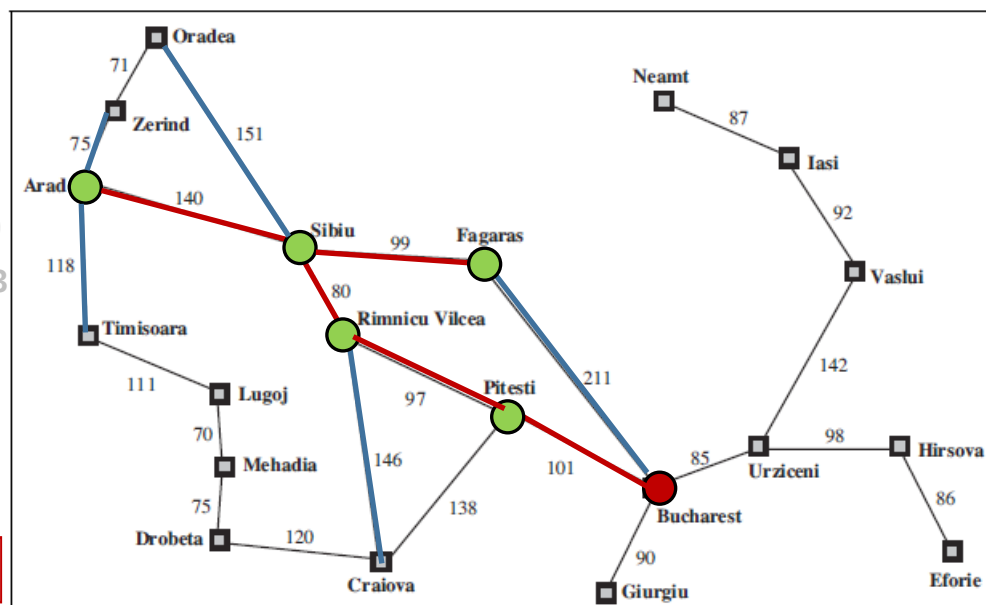


Ex : The Romania Map



QUEUE

[Arad₃₆₆]
 [Sibiu₂₅₃, Timisoara₃₂₉, Zerind₃₇₄]
 [Rim.Vilcea₄₁₃, Fagaras₄₁₅, Timisoara₄₄₇, Zerind₄₄₉, Oradea₆₇₁]
 [Fagaras₄₁₅, Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Craiova₅₂₆, Oradea₆₇₁]
 [Pitesti₄₁₇, Timisoara₄₄₇, Zerind₄₄₉, Bucharest₄₅₀, Craiova₅₂₆, Oradea₆₇₁]
 [Timisoara₄₄₇, Zerind₄₄₉, Craiova₅₂₆, Oradea₆₇₁]



Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Performance Measure for A* Search

- A* search algorithm is **complete** if all step costs exceed some finite $\epsilon > 0$ and if b is finite (like uniform-cost search).
- A* search strategy is **optimal** if the heuristic function $h(n)$ satisfies the following conditions:
 - 1) admissibility
 - 2) consistency (monotonicity)

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Conditions for optimality of A* Search:

- An **admissible** heuristic is one that never overestimates the exact cost to reach the goal.



$$h(n^{(k)}) \leq \sum_{t=2}^k c(n^{(t)}, a_t, n^{(t-1)})$$

for any node $n^{(k)}$ and for any path from $n^{(k)}$ to the goal node $n^{(1)}$.

- If $h(n)$ is admissible, $f(n)$ never overestimates the true cost of a solution along the current path through n .

$$f(n) = g(n) + h(n)$$

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



- If the following condition (triangle inequality) holds, then $h(n)$ is said to be **consistent**:

$$h(n^{(k)}) \leq c(n^{(k)}, a_k, n^{(k-1)}) + h(n^{(k-1)})$$

for every node $n^{(k)}$ and any action a_k .

- Every consistent heuristic is also admissible (**prove it**), but an admissible heuristic rarely may be inconsistent (**give an example**).

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The tree-search version of A^* is **optimal** if $h(n)$ is **admissible**, while the graph-search version is **optimal** if $h(n)$ is **consistent**.

Since every consistent heuristic is also admissible, it is sufficient to prove optimality of A^* for the consistency.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The Proof of Optimality

1. If $h(n)$ is consistent, then the values of $f(n)$ along any path are nondecreasing.

Proof: Suppose node n' is a successor of node n for some action a ; then

$$g(n') = g(n) + c(n, a, n')$$

and we have

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \quad (h(n) \leq c(n, a, n') + h(n')) \\ &\geq g(n) + h(n) = f(n) \\ &\rightarrow f(n') \geq f(n) \end{aligned}$$

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The Proof of Optimality

2. Whenever A^* selects a node n for expansion, the optimal path to that node has been found.

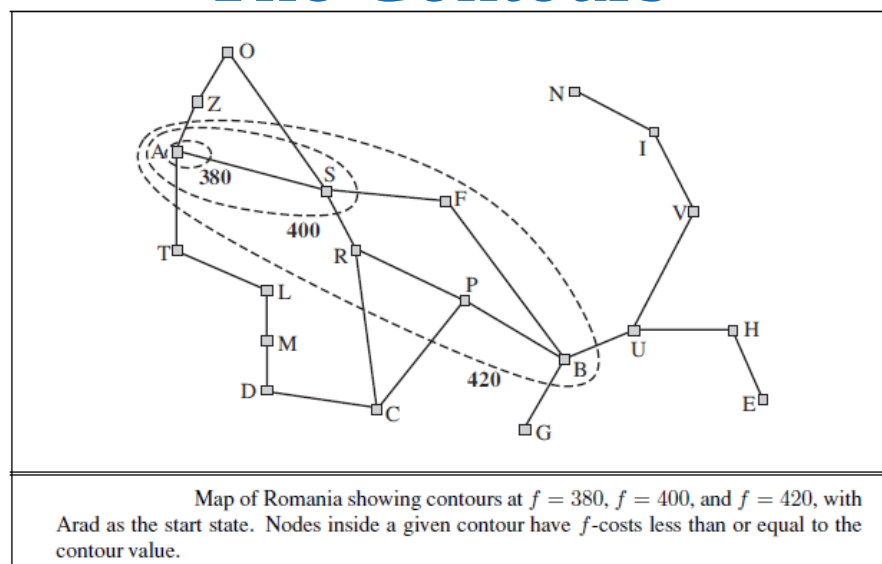
Proof: $f(n)$ along any path is nondecreasing (as proven in step 1) and the algorithm selects minimum cost valued $f(n)$ of a node n for expansion. Therefore, the first goal node selected for expansion must be an optimal solution because because f is the true cost for goal nodes ($h(n) = 0$ at a goal state, thus $f = g$) and other unexplored goal nodes will be at least as expensive.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT

AI

The Contours



The fact that f -costs are nondecreasing along any path also means that we can draw **contours** in the state space. For example, inside the contour labeled 400, all nodes have $f(n)$ less than or equal to 400.

In uniform-cost search
(A* search using $h(n) = 0$)



the contours will be “**circular**” around the start state

In A* search, with
consistent heuristics,



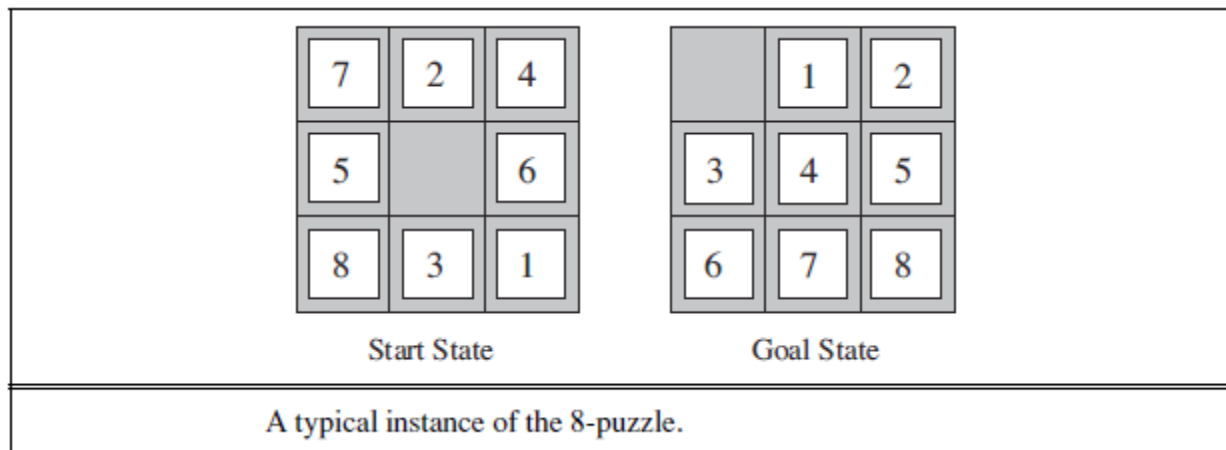
the contours will **stretch toward the goal state** and
become **more narrowly focused around the optimal path**

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Heuristic Functions



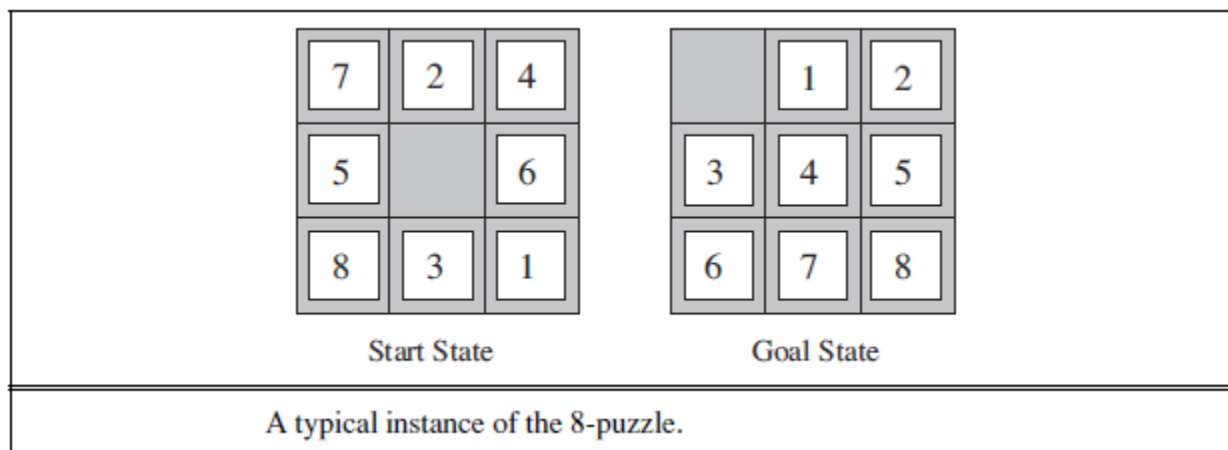
Find appropriate heuristic functions h for the 8-puzzle!!!

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Heuristic Functions



h_1 = the number of misplaced tiles

$$h_1(\text{start state}) = 8$$

$$h_1(\text{goal state}) = 0$$

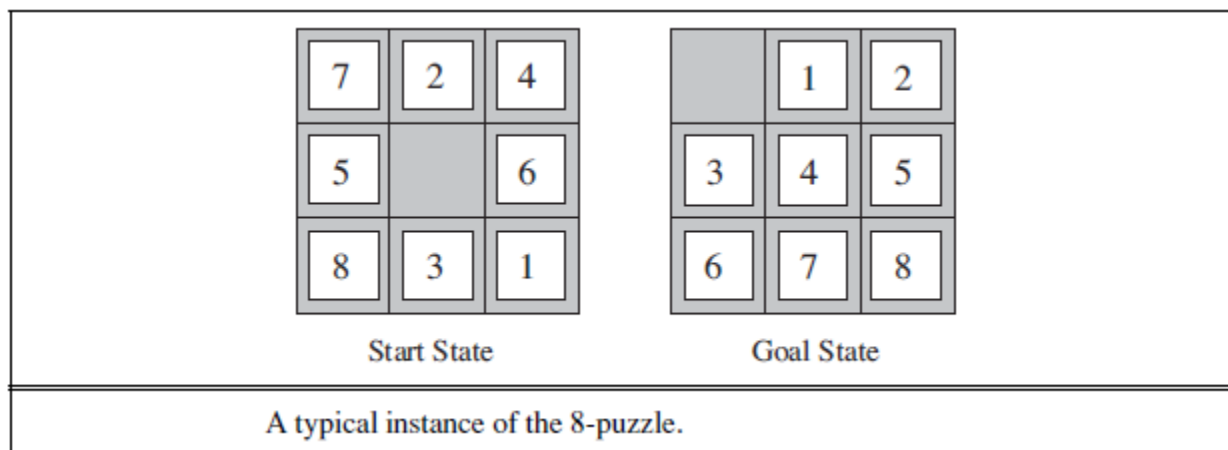
Is h_1 admissible (or consistent) heuristic function? Why?

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Heuristic Functions



h_2 = the sum of the distances of the tiles from their goal positions (Manhattan distance)

$$h_2(\text{start state}) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$$

$$h_2(\text{goal state}) = 0$$

Is h_2 admissible (or consistent) heuristic function? Why?

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The effect of heuristic on performance

- One way to characterize the quality of a heuristic is the **effective branching factor** b^* . If the total number of nodes generated by A* for a particular problem is N and the solution depth is d , then b^* is the branching factor that a uniform tree of depth d would have to have in order to contain $N+1$ nodes. Thus,

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

- For example, if A* finds a solution at depth 5 using 52 nodes, then the effective branching factor is 1.92.
- A well designed heuristic should have a value of b^* close to 1.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The effect of heuristic on performance

	Search Cost (nodes generated)			Effective Branching Factor		
d	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	–	539	113	–	1.44	1.23
16	–	1301	211	–	1.45	1.25
18	–	3056	363	–	1.46	1.26
20	–	7276	676	–	1.47	1.27
22	–	18094	1219	–	1.48	1.28
24	–	39135	1641	–	1.48	1.26

Comparison of the search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A^* algorithms with h_1 , h_2 . Data are averaged over 100 instances of the 8-puzzle for each of various solution lengths d .

To test the heuristic functions h_1 and h_2 , 1200 random problems of 8-puzzle with solution lengths from 2 to 24 (100 for each even number) are generated and are solved with **iterative deepening search** and with **A^* search** using both h_1 and h_2 . The figure shows the average number of nodes generated by each strategy and the effective branching factor.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The effect of heuristic on performance

If $h_2(n) \geq h_1(n)$, we say that h_2 dominates h_1 .

In this case, is h_2 always better than h_1 ?

Yes, because...



If C^* is the cost of the optimal solution path, every node n with $f(n) < C^*$ will surely be expanded ($f(n) = g(n)$ on the goal node). This means that, for all nodes that will be expanded,

$$f(n) = g(n) + h(n) < C^*$$

$$h(n) < C^* - g(n)$$

Since $h_2(n) \geq h_1(n)$ for all nodes, every node that is surely expanded by A* search with h_2 will also surely be expanded with h_1 , and h_1 might cause other nodes to be expanded as well.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The effect of heuristic on performance

It is **generally better** to use a heuristic function **with higher values**, if

- it is consistent and
- the computation time for the heuristic is not too long.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Generating admissible (also consistent) heuristics from relaxed problems

A problem with fewer restrictions on the actions is called a **relaxed problem**.

For example: Consider 8-puzzle

- If the rules of the puzzle were changed so that a tile could move anywhere, even onto an occupied square, instead of just to the adjacent empty square, then the optimal cost of this relaxed problem is:

h_1 = the number of misplaced tiles

- If the rules of the puzzle were changed so that a tile could move one square in any direction, even onto an occupied square, then the optimal cost of this relaxed problem is:

h_2 = Manhattan distance

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Generating admissible (also consistent) heuristics from relaxed problems

The state-space graph of the relaxed problem is a **supergraph** of the original state space because the removal of restrictions creates added edges in the graph.



- Any optimal solution in the original problem is also a solution in the relaxed problem; but the relaxed problem may have better solutions if the added edges provide short cuts. Hence, the cost of an optimal solution to a relaxed problem is an **admissible heuristic** for the original problem.
- Since the derived heuristic is an optimal exact cost for the relaxed problem, it must obey the triangle inequality and therefore it is **consistent heuristic** for the original problem.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Generating admissible (also consistent) heuristics from relaxed problems

If a problem definition is written down in a formal language, it is possible to construct relaxed problems automatically. For example, if the 8-puzzle actions are described as:

A tile can move from square A to square B **if A is horizontally or vertically adjacent to B and B is blank**,

We can generate three relaxed problems by removing one or both of the conditions:

- (a)** A tile can move from square A to square B **if A is horizontally or vertically adjacent to B.**
- (b)** A tile can move from square A to square B **if B is blank.**
- (c)** A tile can move from square A to square B.

From **(a)**, we can derive h_2 (Manhattan Distance).

From **(c)**, we can derive h_1 .

Discuss heuristic (known as Gaschnig's heuristic) derived from **(b)**. Compare with other heuristics.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Generating admissible (also consistent) heuristics from relaxed problems

If a collection of admissible (or consistent) heuristics $h_1, h_2, h_3, \dots, h_m$ is available for a problem and a heuristic h_t , $t \in \{1, 2, \dots, m\}$ dominates any of the others, we can choose h_t . But, if none of them dominates any of the others, then we can define a new heuristic

$$h(n) = \max\{h_1(n), h_2(n), \dots, h_m(n)\}$$

Here for each node, the heuristic that has maximum value is selected. Because the component heuristics are admissible (or consistent), h is also admissible (or consistent). Furthermore, h dominates all of its component heuristics (prove them).

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The generalization of A* Approach

$$f(n) = \alpha g(n) + (1 - \alpha)h(n)$$

Here is α a parameter and $0 \leq \alpha \leq 1$.

- $\alpha=0.5$ $\longrightarrow$ A* search
- $\alpha=1.0$ $\longrightarrow$ Uniform-cost Search
- $\alpha=0.0$ $\longrightarrow$ Greedy Search

The value of α can be changed during the solution.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



More Advanced Algorithms

We have investigated many search algorithms. Apart from these,...

- Different efficient search algorithms can be developed by hybridization of the search algorithms that were investigated.
- Agents can learn new search algorithms using the concept of metalevel state space.
- Agents can learn new heuristic functions from experience.

These are out of scope for this course. You can improve your AI skills by investigating these advanced techniques.

Solving Problems by Searching - 3

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Main Reference

Stuart Russell, Peter Norvig; “Artificial Intelligence A Modern Approach”, Pearson, 3rd Edition, 2013.